Topic 1 - What is DataWeave?

Time to Read: 2 Minutes

What is DataWeave?

DataWeave is MuleSoft's transformation language. It is used to convert data from one format to another and manipulate data as per business requirements.

Simply put:

> If APIs speak different languages, DataWeave acts as the translator.

Why Do We Use It?

You will use DataWeave almost everywhere in MuleSoft projects:

- JSON to JSON transformations.
- XML to JSON conversions.
- CSV processing.
- Filtering data.
- Mapping fields.
- Handling null values.
- Performing calculations.
- Building request and response payloads.

If you know DataWeave well, you can solve most transformation requirements in MuleSoft.

Real-Time Example

Suppose Salesforce sends the following data:

```
{
    "firstName": "Rahul",
    "lastName": "Sharma"
}
```

Your target API expects:

```
{
    "name": "Rahul Sharma"
}
```

DataWeave helps you transform the payload into the required format.

Important Points

- DataWeave is used in almost every MuleSoft project.
- It supports JSON, XML, CSV, Java Objects and many other formats.
- It is one of the most asked MuleSoft interview topics.
- Learning DataWeave will make API transformations much easier.

Interview Tip

If an interviewer asks:

> "What is DataWeave?"

Avoid saying:

> "It is a transformation language."

A better answer is:

> "DataWeave is MuleSoft's powerful transformation language that is used to transform, filter and manipulate data between different formats such as JSON, XML and CSV in real-time integrations."

This answer sounds more practical and project-oriented.

Pro Tip

You don't need to memorize every DataWeave function.

Master the commonly used concepts and functions, and you'll be able to solve most real-world MuleSoft transformation problems.

Topic 2 - Understanding DataWeave Script Structure

Time to Read: 3 Minutes

What is a DataWeave Script?

Every DataWeave transformation follows a simple structure.

```
%dw 2.0
output application/json
---
{
    message: "Hello DataWeave"
}
```

Let's understand each line.

%dw 2.0

```
%dw 2.0
```

- It tells MuleSoft that you're using DataWeave 2.0.
- Every DataWeave script starts with this line.

Think of it as introducing yourself before starting the transformation.

---

output

```
output application/json
```

This tells MuleSoft what format you want in the response.

Some commonly used output types are:

```
output application/json
```

```
output application/xml
```

```
output text/plain
```

```
output application/csv
```

The three dashes (`---`) are extremely important.

Everything written below this line becomes your final output.

Example:

```
%dw 2.0
output application/json
---
"Welcome to DataWeave"
```

Output:

```
"Welcome to DataWeave"
```

Easy (Level 1)

Question

Return a simple welcome message.

Code

```
%dw 2.0
output application/json
---
{
```

```
    message: "Welcome to DataWeave"
}
```

Output

```
{
    "message": "Welcome to DataWeave"
}
```

Medium (Level 2)

Question

Return employee details.

Code

```
%dw 2.0
output application/json
---
{
    employeeId: 101,
    employeeName: "John Doe",
    designation: "MuleSoft Developer"
}
```

Output

```
{
    "employeeId": 101,
    "employeeName": "John Doe",
    "designation": "MuleSoft Developer"
}
```

Difficult (Level 3) - Real-Time Scenario

Question

An API expects customer information in JSON format.

Code

```
%dw 2.0
output application/json
---
{
    customerId: "CUST-1001",
    customerName: "Alice Johnson",
    membershipType: "Premium",
    country: "India"
}
```

Output

```
{
    "customerId": "CUST-1001",
    "customerName": "Alice Johnson",
    "membershipType": "Premium",
    "country": "India"
}
```

Important Points

- `%dw 2.0` is mandatory in every DataWeave script.
- `output` defines the format of the response.
- `---` separates the header section from the actual transformation logic.
- Everything below `---` is returned as output.

Common Mistake

Many beginners forget to add the `output` statement or place their transformation logic above `---`.
Always remember:

```
Header Section
↓
%dw 2.0
output ...

↓

Separator
```

```
---
↓

Transformation Logic
```

Interview Tip

A very common MuleSoft interview question is:

> What is the purpose of `---` in DataWeave?

Answer:

> It separates the DataWeave header (`%dw 2.0` and `output`) from the transformation logic. Everything written below it becomes the final output of the transformation.

Topic 3 - Understanding Input and Output in DataWeave

Time to Read: 3 Minutes

What is Input and Output?

In MuleSoft projects, DataWeave takes data as an input, transforms it, and returns the required output.

Think of it like this:

> Input Data → DataWeave Transformation → Output Data

DataWeave doesn't care where the data comes from. It can transform data from APIs, databases, Salesforce, CSV files, XML files, and much more.

Why is it Important?

Almost every MuleSoft integration follows this pattern:

- Receive data.
- Transform data.
- Send data to another system.

If you understand input and output, understanding DataWeave becomes much easier.

Easy (Level 1)

Question

Convert a JSON payload into a simple response.

Input

```
{
    "name": "John Doe"
}
```

Code

```
%dw 2.0
output application/json
---
{
    message: payload.name
}
```

Output

```
{
    "message": "John Doe"
}
```

Medium (Level 2)

Question

Return only the required employee details.

Input

```
{
    "employeeId": 101,
    "employeeName": "Michael Smith",
    "salary": 80000
}
```

Code

```
%dw 2.0
output application/json
---
{
    id: payload.employeeId,
    name: payload.employeeName
}
```

Output

```
{
    "id": 101,
    "name": "Michael Smith"
}
```

Difficult (Level 3) - Real-Time Scenario

Question

Your API receives customer information and needs to send a simplified response to another system.

Input

```
{
    "customerId": "CUST-1001",
    "customerName": "Emma Wilson",
    "membershipType": "Premium",
    "country": "India",
    "status": "Active"
}
```

Code

```
%dw 2.0
output application/json
---
{
    customerId: payload.customerId,
    customerName: payload.customerName,
    status: payload.status
}
```

Output

```
{
    "customerId": "CUST-1001",
    "customerName": "Emma Wilson",
    "status": "Active"
}
```

Important Points

- `payload` represents the incoming data in MuleSoft.
- DataWeave reads the input and produces the required output.
- You don't always need to return the complete payload.
- Transform only the fields that are required.

Common Mistake

Many beginners think DataWeave automatically returns the entire payload.

This is incorrect.

DataWeave only returns what you explicitly write below the `---` section.

Interview Tip

A very common interview question is:

> What is `payload` in MuleSoft?

Answer:

> `payload` is the data currently being processed in a Mule event. DataWeave uses the payload to read and transform incoming data into the required format.

Pro Tip

If you understand how to work with `payload`, you're already halfway to mastering DataWeave. Almost every DataWeave transformation in MuleSoft revolves around reading, modifying, and returning payload data.

Topic 4 - Data Types in DataWeave

Time to Read: 4 Minutes

What are Data Types?

Data Types define what kind of value you are working with.

The most commonly used Data Types in DataWeave are:

- String
- Number
- Boolean
- Array
- Object
- Null
- Date
- DateTime

You don't need to memorize all Data Types on day one. Master the commonly used ones first.

Why Do We Need Them?

Different DataWeave functions work with different Data Types.

For example:

- `upper()` works with Strings.
- `map()` works with Arrays.
- `mapObject()` works with Objects.
- Mathematical operations work with Numbers.

Understanding Data Types helps you avoid many transformation errors.

Common Data Types

Data Type	Example
String	"John Doe"
Number	1001
Boolean	true

Data Type	Example
Array	["Java", "MuleSoft"]
Object	{"name":"John"}
Null	null
Date	2026-07-16
DateTime	2026-07-16T10:30:00

Easy (Level 1)

Question

Return a customer's name and age.

Code

```
%dw 2.0
output application/json
---
{
    name: "John Doe",
    age: 28
}
```

Output

```
{
    "name": "John Doe",
    "age": 28
}
```

Medium (Level 2)

Question

Return employee details using multiple Data Types.

Code

```
%dw 2.0
output application/json
---
{
    employeeId: 101,
    employeeName: "Michael Smith",
    isActive: true,
    skills: ["MuleSoft", "DataWeave"]
}
```

Output

```
{
    "employeeId": 101,
    "employeeName": "Michael Smith",
    "isActive": true,
    "skills": [
        "MuleSoft",
        "DataWeave"
    ]
}
```

Difficult (Level 3) - Real-Time Scenario

Question

An API needs customer information along with membership details and purchase history.

Code

```
%dw 2.0
output application/json
---
{
    customerId: "CUST-1001",
    customerName: "Emma Wilson",
    premiumMember: true,
    totalOrders: 15,
    purchasedItems: [
        "Laptop",
        "Keyboard",
```

```
        "Mouse"
    ]
}
```

Output

```
{
    "customerId": "CUST-1001",
    "customerName": "Emma Wilson",
    "premiumMember": true,
    "totalOrders": 15,
    "purchasedItems": [
        "Laptop",
        "Keyboard",
        "Mouse"
    ]
}
```

Important Points

- Arrays use square brackets `[]`.
- Objects use curly braces `{}`.
- Strings are enclosed in double quotes.
- Numbers do not use quotes.
- Boolean values are written as `true` or `false`.
- Choosing the correct Data Type is extremely important while transforming data.

Common Mistake

Many developers try to use String functions on Arrays or Object functions on Strings.

Always ask yourself:

> What Data Type am I working with?

Knowing the answer will immediately tell you which DataWeave functions can be used.

Interview Tip

A common interview question is:

> Which Data Types do you use most often in DataWeave?

A good answer is:

> String, Number, Boolean, Array, Object, Null, Date, and DateTime are the most commonly used Data Types in real MuleSoft projects.

Pro Tip

Before writing any DataWeave transformation, spend a few seconds understanding the input payload. Identifying the Data Types correctly will save you a lot of debugging time later.

Topic 5 - Variables in DataWeave (var)

Time to Read: 3 Minutes

What are Variables?

Variables are used to store values that you want to reuse in your DataWeave script.

Instead of writing the same value multiple times, you can store it in a variable and use it wherever required.

Think of variables as temporary containers for your data.

Why Do We Need Variables?

Variables help you:

- Write cleaner code.
- Avoid repeating values.
- Make transformations easier to understand.
- Improve code readability and maintainability.

Syntax

```
var variableName = value
```

Variables are declared before the `---` separator.

Easy (Level 1)

Question

Store an employee's name in a variable and return it.

Code

```
%dw 2.0
output application/json
var employeeName = "John Doe"
```

```
---
{
    name: employeeName
}
```

Output

```
{
    "name": "John Doe"
}
```

Medium (Level 2)

Question

Store multiple employee details in variables.

Code

```
%dw 2.0
output application/json
var employeeId = 101
var designation = "MuleSoft Developer"
---
{
    id: employeeId,
    role: designation
}
```

Output

```
{
    "id": 101,
    "role": "MuleSoft Developer"
}
```

Difficult (Level 3) - Real-Time Scenario

Question

Your API receives customer details. Store the customer's membership status in a variable and use it while building the response payload.

Input

```
{
    "customerName": "Emma Wilson",
    "membershipType": "Premium"
}
```

Code

```
%dw 2.0
output application/json
var membership = payload.membershipType
---
{
    customerName: payload.customerName,
    membershipStatus: membership,
    welcomeMessage: "Premium Member"
}
```

Output

```
{
    "customerName": "Emma Wilson",
    "membershipStatus": "Premium",
    "welcomeMessage": "Premium Member"
}
```

Important Points

- Variables are declared before the `---` separator.
- They can store Strings, Numbers, Arrays, Objects, and even complex expressions.
- Variables improve readability and reduce duplicate code.
- They are commonly used in real-world MuleSoft projects.

Common Mistake

Many beginners try to declare variables below the `---` separator.

Remember:

```
%dw 2.0
output ...

var variables go here

---

Transformation logic goes here.
```

Interview Tip

A common interview question is:

> Why do we use variables in DataWeave?

A good answer is:

> Variables help make DataWeave scripts more readable, reusable, and easier to maintain by avoiding duplicate logic and storing intermediate values.

Pro Tip

Whenever you find yourself writing the same expression multiple times, consider storing it in a variable. Your transformations will become much cleaner and easier to debug.

Topic 6 - Operators in DataWeave

Time to Read: 4 Minutes

What are Operators?

Operators are used to perform operations on data.

In DataWeave, the most commonly used operators are:

- Arithmetic Operators
- Comparison Operators
- Logical Operators
- Conditional Operators

Don't try to memorize all of them. Focus on understanding where and when to use them.

Arithmetic Operators

Used for mathematical calculations.

Operator	Example
+	10 + 5
-	10 - 5
*	10 * 5
/	10 / 5
%	10 % 3

Comparison Operators

Used for comparing values.

Operator	Example
==	age == 25
!=	age != 25
>	salary > 50000

Operator	Example
<	salary < 50000
>=	marks >= 70
<=	marks <= 70

Logical Operators

Used when multiple conditions are involved.

Operator	Example
and	condition1 and condition2
or	condition1 or condition2
not	not(condition)

Conditional Operator

Used to make decisions.

```
if(condition)
   value1
else
   value2
```

Easy (Level 1)

Question

Calculate the total amount of an order.

Code

```
%dw 2.0
output application/json
---
{
```

```
    totalAmount: 1000 + 500
}
```

Output

```
{
    "totalAmount": 1500
}
```

Medium (Level 2)

Question

Check whether an employee is eligible for a bonus.

Input

```
{
    "salary": 85000
}
```

Code

```
%dw 2.0
output application/json
---
{
    bonusEligible:
        if(payload.salary > 80000)
            true
        else
            false
}
```

Output

```
{
    "bonusEligible": true
}
```

Difficult (Level 3) - Real-Time Scenario

Question

An API receives customer details. Premium customers from India should receive a special discount.

Input

```
{
    "customerType": "Premium",
    "country": "India"
}
```

Code

```
%dw 2.0
output application/json
---
{
    specialDiscount:
        if(
            payload.customerType == "Premium"
            and
            payload.country == "India"
        )
            "20% Discount"
        else
            "No Discount"
}
```

Output

```
{
    "specialDiscount": "20% Discount"
}
```

Important Points

- Arithmetic operators are used for calculations.
- Comparison operators are used for checking values.
- Logical operators are useful when multiple conditions are involved.
- Conditional logic is heavily used in DataWeave transformations.

Common Mistake

Many beginners write:

```
if(payload.salary = 50000)
```

This is incorrect.

Always use:

```
if(payload.salary == 50000)
```

Remember:

- `=` is used for variable assignment.
- `==` is used for comparison.

Interview Tip

A commonly asked interview question is:

> Which operator is most frequently used in DataWeave transformations?

A practical answer is:

> Comparison operators and conditional statements are used extensively in real-world MuleSoft projects for applying business rules and filtering data.

Pro Tip

If you're writing DataWeave transformations for APIs, you'll use `if-else`, `==`, `>`, `and`, and `or` almost every day. Master these before moving to advanced functions.

Topic 7 - Conditional Statements (if-else)

Time to Read: 3 Minutes

What are Conditional Statements?

Conditional statements help DataWeave make decisions.

Whenever your transformation depends on certain conditions, you'll use `if-else`.

Think of it as asking a simple question:

> If this condition is true, do one thing. Otherwise, do something else.

Syntax

```
if(condition)
    value1
else
    value2
```

Where Do We Use It?

Some common real-world use cases are:

- Checking customer membership status.
- Applying discounts.
- Setting default values.
- Validating business rules.
- Returning different API responses.

Easy (Level 1)

Question

Return whether a customer is an adult or minor.

Input

```
{
    "age": 22
}
```

Code

```
%dw 2.0
output application/json
---
{
    category:
        if (payload.age >= 18)
            "Adult"
        else
            "Minor"
}
```

Output

```
{
    "category": "Adult"
}
```

Medium (Level 2)

Question

Return the membership level of a customer.

Input

```
{
    "totalPurchases": 75000
}
```

Code

```
%dw 2.0
output application/json
---
{
    membership:
        if (payload.totalPurchases >= 50000)
            "Premium"
        else
            "Standard"
}
```

Output

```
{
    "membership": "Premium"
}
```

Difficult (Level 3) - Real-Time Scenario

Question

An Order API receives the total order amount. Orders above 10,000 are eligible for free delivery.

Input

```
{
    "orderAmount": 12500
}
```

Code

```
%dw 2.0
output application/json
---
{
    orderAmount: payload.orderAmount,
    deliveryCharge:
        if (payload.orderAmount >= 10000)
            0
```

```
        else
            150,
    deliveryType:
        if (payload.orderAmount >= 10000)
            "Free Delivery"
        else
            "Standard Delivery"
}
```

Output

```
{
    "orderAmount": 12500,
    "deliveryCharge": 0,
    "deliveryType": "Free Delivery"
}
```

Important Points

- `if-else` is used to apply business logic.
- Conditions can use comparison and logical operators.
- It can return Strings, Numbers, Objects, Arrays, or Boolean values.
- It is one of the most commonly used concepts in DataWeave.

Common Mistake

Many beginners forget that DataWeave is expression-based.

This is perfectly valid:

```
discount:
    if (payload.customerType == "Premium")
        "20%"
    else
        "5%"
```

The `if-else` statement itself returns a value.

Interview Tip

A common interview question is:

> Can we use `if-else` inside an Object in DataWeave?

Answer:

> Yes. In fact, it is one of the most common ways of applying business logic while building API request and response payloads.

Pro Tip

Before learning advanced functions like `filter()` and `map()`, become comfortable with `if-else`. You'll often use conditional logic along with these functions in real-world MuleSoft transformations.

Topic 8 - Arrays in DataWeave

Time to Read: 4 Minutes

What is an Array?

An Array is simply a collection of values stored inside square brackets `[]`.

You can store:

- Strings
- Numbers
- Objects
- Boolean values
- Even other Arrays

In real MuleSoft projects, APIs often send multiple records in the form of Arrays.

Where Do We Use Arrays?

You'll frequently work with Arrays when dealing with:

- Customer records
- Employee data
- Orders
- Products
- API responses
- Salesforce records
- CSV data

Most DataWeave functions like `map()`, `filter()`, `distinctBy()`, `orderBy()`, and `reduce()` work with Arrays.

Array Example

```
[
    "Laptop",
    "Keyboard",
    "Mouse"
]
```

Another example:

```
[
    {
        "employeeId": 101,
        "name": "John Doe"
    },
    {
        "employeeId": 102,
        "name": "Emma Wilson"
    }
]
```

Easy (Level 1)

Question

Return a list of programming languages.

Code

```
%dw 2.0
output application/json
---
[
    "Java",
    "MuleSoft",
    "DataWeave"
]
```

Output

```
[
    "Java",
    "MuleSoft",
    "DataWeave"
]
```

Medium (Level 2)

Question

Return a list of employee names.

Code

```
%dw 2.0
output application/json
---
[
    {
        name: "John Doe"
    },
    {
        name: "Michael Smith"
    }
]
```

Output

```
[
    {
        "name": "John Doe"
    },
    {
        "name": "Michael Smith"
    }
]
```

Difficult (Level 3) - Real-Time Scenario

Question

An API returns multiple customer records. Your transformation should return the customer details as an Array.

Code

```
%dw 2.0
output application/json
```

```
---
[
    {
        customerId: "CUST-1001",
        customerName: "Alice Johnson",
        membershipType: "Premium"
    },
    {
        customerId: "CUST-1002",
        customerName: "David Miller",
        membershipType: "Standard"
    }
]
```

Output

```
[
    {
        "customerId": "CUST-1001",
        "customerName": "Alice Johnson",
        "membershipType": "Premium"
    },
    {
        "customerId": "CUST-1002",
        "customerName": "David Miller",
        "membershipType": "Standard"
    }
]
```

Important Points

- Arrays are enclosed within square brackets `[]`.
- Arrays can contain one or more values.
- Most DataWeave transformation functions work with Arrays.
- APIs commonly return multiple records as Arrays.

Common Mistake

Many beginners confuse Arrays with Objects.

Remember:

```
{
    "name": "John Doe"
}
```

This is an Object.

```
[
    "John Doe",
    "Emma Wilson"
]
```

This is an Array.

Interview Tip

A very common interview question is:

> Which DataWeave functions are commonly used with Arrays?

A good answer is:

> `map()`, `filter()`, `reduce()`, `distinctBy()`, `groupBy()`, `orderBy()`, and `flatten()` are some of the most commonly used Array functions in MuleSoft projects.

Pro Tip

Whenever you see multiple records in an API response, there's a very high chance you'll be using Array functions to transform them. Master Arrays first—learning DataWeave functions will become much easier.

Topic 9 - Objects in DataWeave

Time to Read: 3 Minutes

What is an Object?

An Object is a collection of key-value pairs enclosed within curly braces `{}`.

In simple terms:

> Key = Field Name
> Value = Actual Data

Most API requests and responses are represented as Objects in MuleSoft.

Where Do We Use Objects?

You'll work with Objects almost every day in MuleSoft projects, such as:

- Customer details
- Employee records
- API request payloads
- API response payloads
- Salesforce records
- Payment information
- Order details

Object Example

```
{
    "customerId": "CUST-1001",
    "customerName": "John Doe",
    "country": "India"
}
```

Here:

- `customerId` is the key.
- `"CUST-1001"` is the value.

Easy (Level 1)

Question

Create an Object containing product details.

Code

```
%dw 2.0
output application/json
---
{
    productId: "P-1001",
    productName: "Wireless Mouse",
    price: 899
}
```

Output

```
{
    "productId": "P-1001",
    "productName": "Wireless Mouse",
    "price": 899
}
```

Medium (Level 2)

Question

Create an Object containing employee information.

Code

```
%dw 2.0
output application/json
---
{
    employeeId: 101,
    employeeName: "Emma Wilson",
    designation: "Integration Developer",
```

```
    experience: 5
}
```

Output

```
{
    "employeeId": 101,
    "employeeName": "Emma Wilson",
    "designation": "Integration Developer",
    "experience": 5
}
```

Difficult (Level 3) - Real-Time Scenario

Question

An API expects customer information in the following format.

Code

```
%dw 2.0
output application/json
---
{
    customerId: "CUST-2001",
    customerName: "Michael Smith",
    membershipType: "Premium",
    totalOrders: 25,
    emailVerified: true
}
```

Output

```
{
    "customerId": "CUST-2001",
    "customerName": "Michael Smith",
    "membershipType": "Premium",
    "totalOrders": 25,
    "emailVerified": true
}
```

Important Points

- Objects are enclosed within curly braces `{}`.
- Objects consist of key-value pairs.
- Most API payloads in MuleSoft are Objects.
- Objects can contain Strings, Numbers, Arrays, Booleans, and even other Objects.

Nested Object Example

```
{
    "customerId": "CUST-1001",
    "address": {
        "city": "Pune",
        "country": "India"
    }
}
```

Nested Objects are extremely common in real-world integrations.

Common Mistake

Many beginners assume that Arrays and Objects are the same.

Remember:

```
{
    "name": "John Doe"
}
```

This is an Object.

```
[
    {
        "name": "John Doe"
    },
    {
        "name": "Emma Wilson"
    }
]
```

This is an Array containing two Objects.

Interview Tip

A common MuleSoft interview question is:

> Can an Object contain Arrays and other Objects?

Answer:

> Yes. DataWeave Objects can contain Arrays, nested Objects, Numbers, Strings, Booleans, and other supported Data Types. Nested payloads are very common in real-world MuleSoft projects.

Pro Tip

Before writing any transformation, identify whether the payload is:

- An Object
- An Array
- An Array of Objects

This simple habit will help you immediately decide which DataWeave functions are appropriate for the transformation.

Topic 10 - Payload and Accessing Fields

Time to Read: 3 Minutes

What is Payload?

In MuleSoft, `payload` represents the data that is currently being processed.

Whenever an API sends data to your Mule application, it becomes the payload.

Think of it as:

> Payload = Incoming Data

Almost every DataWeave transformation starts by reading values from the payload.

How Do We Access Fields?

We use dot notation (`.`) to access fields.

Example:

```
payload.customerName
```

If the payload is:

```
{
    "customerName": "John Doe"
}
```

Then:

```
payload.customerName
```

returns:

```
John Doe
```

Easy (Level 1)

Question

Return only the employee name from the payload.

Input

```
{
    "employeeName": "Emma Wilson"
}
```

Code

```
%dw 2.0
output application/json
---
{
    name: payload.employeeName
}
```

Output

```
{
    "name": "Emma Wilson"
}
```

Medium (Level 2)

Question

Return selected customer details.

Input

```
{
    "customerId": "CUST-1001",
    "customerName": "Michael Smith",
    "country": "India"
}
```

Code

```
%dw 2.0
output application/json
---
{
    id: payload.customerId,
    name: payload.customerName
}
```

Output

```
{
    "id": "CUST-1001",
    "name": "Michael Smith"
}
```

Difficult (Level 3) - Real-Time Scenario

Question

An Order API returns the following payload. Return only the fields required by another downstream system.

Input

```
{
    "orderId": "ORD-101",
    "customerName": "Alice Johnson",
    "orderAmount": 15000,
    "paymentStatus": "Success",
    "deliveryStatus": "Dispatched"
}
```

Code

```
%dw 2.0
output application/json
---
{
    orderId: payload.orderId,
    customerName: payload.customerName,
```

```
    paymentStatus: payload.paymentStatus
}
```

Output

```
{
    "orderId": "ORD-101",
    "customerName": "Alice Johnson",
    "paymentStatus": "Success"
}
```

Nested Field Access

Real-world APIs usually contain nested Objects.

Input

```
{
    "customer": {
        "name": "David Miller",
        "country": "India"
    }
}
```

To access the customer's name:

```
payload.customer.name
```

Output:

```
David Miller
```

Important Points

- `payload` represents the incoming data.
- Dot notation is used to access fields.
- Nested fields can be accessed using multiple dots.
- Most DataWeave transformations read values from the payload.

Common Mistake

Beginners often write:

```
payload.customer-name
```

This is incorrect.

Correct:

```
payload.customerName
```

If the field name contains spaces or special characters, use:

```
payload."customer-name"
```

Interview Tip

A common interview question is:

> How do you access nested fields in DataWeave?

Answer:

> Nested fields are accessed using dot notation. For example, `payload.customer.address.city`.

Pro Tip

Whenever you're working on a MuleSoft project, spend a few seconds understanding the payload structure before writing any DataWeave code. Knowing where the data lives makes transformations much easier.

Topic 11 - Arrays of Objects

Time to Read: 4 Minutes

What are Arrays of Objects?

An Array of Objects is simply a collection of multiple Objects stored inside an Array.

In MuleSoft projects, this is one of the most common payload structures you'll encounter.

Example:

```
[
    {
        "customerId": "CUST-1001",
        "customerName": "John Doe"
    },
    {
        "customerId": "CUST-1002",
        "customerName": "Emma Wilson"
    }
]
```

Think of it as:

> Multiple records = Array of Objects

Where Do We Use It?

You'll see Arrays of Objects in:

- Customer APIs
- Employee records
- Order details
- Salesforce responses
- Product catalogs
- Database records
- CSV transformations

Most DataWeave Array functions work on Arrays of Objects.

Easy (Level 1)

Question

Return a list of products.

Code

```
%dw 2.0
output application/json
---
[
    {
        productId: "P-101",
        productName: "Laptop"
    },
    {
        productId: "P-102",
        productName: "Keyboard"
    }
]
```

Output

```
[
    {
        "productId": "P-101",
        "productName": "Laptop"
    },
    {
        "productId": "P-102",
        "productName": "Keyboard"
    }
]
```

Medium (Level 2)

Question

An API returns employee details in an Array.

Input

```
[
    {
        "employeeId": 101,
        "employeeName": "Michael Smith"
    },
    {
        "employeeId": 102,
        "employeeName": "Alice Johnson"
    }
]
```

Accessing Values

```
payload[0].employeeName
```

Output:

```
Michael Smith
```

```
payload[1].employeeName
```

Output:

```
Alice Johnson
```

Difficult (Level 3) - Real-Time Scenario

Question

An Order API returns multiple orders. The downstream system expects the same structure.

Code

```
%dw 2.0
output application/json
---
[
```

```
    {
        orderId: "ORD-1001",
        amount: 5000,
        paymentStatus: "Success"
    },
    {
        orderId: "ORD-1002",
        amount: 8500,
        paymentStatus: "Pending"
    },
    {
        orderId: "ORD-1003",
        amount: 12000,
        paymentStatus: "Success"
    }
]
```

Output

```
[
    {
        "orderId": "ORD-1001",
        "amount": 5000,
        "paymentStatus": "Success"
    },
    {
        "orderId": "ORD-1002",
        "amount": 8500,
        "paymentStatus": "Pending"
    },
    {
        "orderId": "ORD-1003",
        "amount": 12000,
        "paymentStatus": "Success"
    }
]
```

Accessing Array Elements

Expression	Meaning
payload[0]	First Object
payload[1]	Second Object

Expression	Meaning
payload[2]	Third Object

Examples:

```
payload[0].orderId
```

```
payload[1].amount
```

```
payload[2].paymentStatus
```

Important Points

- Arrays use square brackets `[]`.
- Objects use curly braces `{}`.
- Arrays of Objects are extremely common in MuleSoft APIs.
- Array indexing starts from 0.
- Most DataWeave functions operate on Arrays of Objects.

Common Mistake

Many beginners forget that:

```
payload.customerName
```

works only when the payload is an Object.

If the payload is an Array of Objects, this will not work.

Instead, use:

```
payload[0].customerName
```

or use functions like `map()` when working with multiple records.

Interview Tip

A common interview question is:

> What is the most common payload structure you work with in MuleSoft projects?

A practical answer is:

> Arrays of Objects are one of the most common payload structures because APIs frequently return multiple records such as customers, employees, orders, and products.

Pro Tip

Whenever you receive an API response, first determine whether the payload is:

- An Object
- An Array
- An Array of Objects

This habit will help you immediately decide which DataWeave functions to use for the transformation.

Topic 12 - map()

Time to Read: 4 Minutes

What does it do?

`map()` is used to iterate through every item in an Array and transform it into a new value.

Think of it as:

> "Take one item at a time, do something with it, and return a new Array."

When do we use it?

You'll use `map()` when:

- Transforming API responses.
- Building request payloads.
- Renaming fields.
- Performing calculations.
- Formatting data before sending it to another system.

Syntax

```
payload map (item) -> item
```

You can also use the shortcut:

```
payload map $
```

where `$` represents the current item.

Important Points

- `map()` works only with Arrays.
- It always returns a new Array.
- It is one of the most commonly used DataWeave functions.
- It is heavily used in almost every MuleSoft project.

Easy (Level 1)

Question

Convert all employee names to uppercase.

Input

```
[
    "john",
    "emma",
    "michael"
]
```

Code

```
%dw 2.0
output application/json
---
payload map upper($)
```

Output

```
[
    "JOHN",
    "EMMA",
    "MICHAEL"
]
```

Medium (Level 2)

Question

Return only the employee ID and employee name.

Input

```
[
    {
        "employeeId": 101,
```

```
        "employeeName": "John Doe",
        "salary": 80000
    },
    {
        "employeeId": 102,
        "employeeName": "Emma Wilson",
        "salary": 90000
    }
]
```

Code

```
%dw 2.0
output application/json
---
payload map {
    id: $.employeeId,
    name: $.employeeName
}
```

Output

```
[
    {
        "id": 101,
        "name": "John Doe"
    },
    {
        "id": 102,
        "name": "Emma Wilson"
    }
]
```

Difficult (Level 3) - Real-Time Scenario

Question

A Customer API returns customer details. The downstream system expects a simplified payload with a welcome message.

Input

```
[
    {
        "customerId": "CUST-1001",
        "customerName": "Alice Johnson",
        "membershipType": "Premium"
    },
    {
        "customerId": "CUST-1002",
        "customerName": "David Miller",
        "membershipType": "Standard"
    }
]
```

Code

```
%dw 2.0
output application/json
---
payload map {
    customerId: $.customerId,
    customerName: $.customerName,
    welcomeMessage:
        if ($.membershipType == "Premium")
            "Welcome Premium Customer"
        else
            "Welcome Customer"
}
```

Output

```
[
    {
        "customerId": "CUST-1001",
        "customerName": "Alice Johnson",
        "welcomeMessage": "Welcome Premium Customer"
    },
    {
        "customerId": "CUST-1002",
        "customerName": "David Miller",
        "welcomeMessage": "Welcome Customer"
    }
]
```

Common Mistake

Many beginners try to use `map()` on an Object.

Incorrect:

```
payload map ...
```

when the payload is:

```
{
    "name": "John Doe"
}
```

Remember:

> `map()` works only with Arrays.

For Objects, you'll learn `mapObject()` later in the eBook.

Interview Tip

A very common interview question is:

> What is the difference between `map()` and `mapObject()`?

A simple answer is:

> `map()` is used for Arrays, whereas `mapObject()` is used for Objects.

Pro Tip

If I had to pick only one DataWeave function to master first, it would be `map()`. Once you're comfortable with `map()`, learning `filter()`, `groupBy()`, `distinctBy()`, and `reduce()` becomes much easier.

Topic 13 - filter()

Time to Read: 4 Minutes

What does it do?

`filter()` is used to remove unwanted records from an Array.

Think of it as:

> "Keep only the records that match my condition."

When do we use it?

You'll use `filter()` when:

- Returning only active customers.
- Removing failed transactions.
- Filtering premium members.
- Selecting successful orders.
- Applying business rules on API responses.

Syntax

```
payload filter (item) -> condition
```

You can also use the shortcut:

```
payload filter (condition)
```

where `$` represents the current item.

Important Points

- `filter()` works only with Arrays.
- It always returns an Array.
- If no records match the condition, an empty Array is returned.
- It is commonly used with `map()` in MuleSoft projects.

Easy (Level 1)

Question

Return only the even numbers.

Input

```
[
    10,
    15,
    20,
    25,
    30
]
```

Code

```
%dw 2.0
output application/json
---
payload filter ($ mod 2 == 0)
```

Output

```
[
    10,
    20,
    30
]
```

Medium (Level 2)

Question

Return only active employees.

Input

```
[
    {
        "employeeId": 101,
        "employeeName": "John Doe",
        "isActive": true
    },
    {
        "employeeId": 102,
        "employeeName": "Emma Wilson",
        "isActive": false
    }
]
```

Code

```
%dw 2.0
output application/json
---
payload filter ($.isActive == true)
```

Output

```
[
    {
        "employeeId": 101,
        "employeeName": "John Doe",
        "isActive": true
    }
]
```

Difficult (Level 3) - Real-Time Scenario

Question

An Order API returns multiple orders. Return only the successfully paid orders.

Input

```
[
    {
        "orderId": "ORD-1001",
        "amount": 12000,
        "paymentStatus": "Success"
    },
    {
        "orderId": "ORD-1002",
        "amount": 8000,
        "paymentStatus": "Failed"
    },
    {
        "orderId": "ORD-1003",
        "amount": 15000,
        "paymentStatus": "Success"
    }
]
```

Code

```
%dw 2.0
output application/json
---
payload filter ($.paymentStatus == "Success")
```

Output

```
[
    {
        "orderId": "ORD-1001",
        "amount": 12000,
        "paymentStatus": "Success"
    },
    {
        "orderId": "ORD-1003",
        "amount": 15000,
        "paymentStatus": "Success"
    }
]
```

Common Mistake

Many developers expect `filter()` to return an Object.

Remember:

> `filter()` always returns an Array.

Even if only one record matches the condition, the result will still be an Array.

Interview Tip

A commonly asked interview question is:

> What happens if no records satisfy the filter condition?

Answer:

> `filter()` returns an empty Array (`[]`). It does not throw an exception.

Pro Tip

In real-world MuleSoft projects, you'll frequently use:

```
filter() + map()
```

A common approach is:

1. Filter the records you need.
2. Transform them using `map()`.
3. Return the final payload.

This combination solves a large number of API transformation requirements.

Topic 14 - Using map() and filter() Together

Time to Read: 5 Minutes

Why Should You Learn This?

In real-world MuleSoft projects, you'll rarely use just one DataWeave function.

A very common approach is:

> Filter the records you need → Transform them using map() → Return the final payload.

This pattern is used extensively while working with:

- Customer records
- Orders
- Payments
- Employee data
- Salesforce API responses
- External system integrations

How Does It Work?

```
Input Payload
      ↓
   filter()
(Remove unwanted records)
      ↓
    map()
(Transform required fields)
      ↓
 Final Output
```

Easy (Level 1)

Question

Return only even numbers and multiply them by 10.

Input

```
[
    10,
    15,
    20,
    25
]
```

Code

```
%dw 2.0
output application/json
---
payload
    filter ($ mod 2 == 0)
    map ($ * 10)
```

Output

```
[
    100,
    200
]
```

Medium (Level 2)

Question

Return only active employees and display their names.

Input

```
[
    {
        "employeeId": 101,
        "employeeName": "John Doe",
        "isActive": true
    },
    {
        "employeeId": 102,
```

```
        "employeeName": "Emma Wilson",
        "isActive": false
    },
    {
        "employeeId": 103,
        "employeeName": "Michael Smith",
        "isActive": true
    }
]
```

Code

```
%dw 2.0
output application/json
---
payload
    filter ($.isActive)
    map {
        name: $.employeeName
    }
```

Output

```
[
    {
        "name": "John Doe"
    },
    {
        "name": "Michael Smith"
    }
]
```

Difficult (Level 3) - Real-Time Scenario

Question

A Customer API returns customer records. Your downstream system requires:

- Only Premium customers.
- Return customer ID and customer name.
- Add a welcome message.

Input

```
[
    {
        "customerId": "CUST-1001",
        "customerName": "Alice Johnson",
        "membershipType": "Premium"
    },
    {
        "customerId": "CUST-1002",
        "customerName": "David Miller",
        "membershipType": "Standard"
    },
    {
        "customerId": "CUST-1003",
        "customerName": "Emma Wilson",
        "membershipType": "Premium"
    }
]
```

Code

```
%dw 2.0
output application/json
---
payload
    filter ($.membershipType == "Premium")
    map {
        customerId: $.customerId,
        customerName: $.customerName,
        message: "Welcome Premium Customer"
    }
```

Output

```
[
    {
        "customerId": "CUST-1001",
        "customerName": "Alice Johnson",
        "message": "Welcome Premium Customer"
    },
    {
        "customerId": "CUST-1003",
        "customerName": "Emma Wilson",
```

```
        "message": "Welcome Premium Customer"
    }
]
```

Important Points

- `filter()` is usually performed before `map()`.
- `filter()` removes unwanted records.
- `map()` transforms the remaining records.
- This combination is extremely common in MuleSoft integrations.

Common Mistake

Incorrect approach:

```
payload map (...) filter (...)
```

Recommended approach:

```
payload
    filter (...)
    map (...)
```

Filtering first improves readability and performance by transforming only the required records.

Interview Tip

A commonly asked interview question is:

> Which DataWeave functions are most commonly used together in real projects?

A practical answer is:

> `filter()` and `map()` are frequently used together to filter API responses and transform the required data into the expected format.

Pro Tip

If you master:

- map()
- filter()
- if-else
- Variables
- Payload handling

you can already solve a large percentage of DataWeave transformation requirements in MuleSoft projects.

Topic 15 - mapObject()

Time to Read: 4 Minutes

What does it do?

`mapObject()` is used to iterate through an Object and transform its key-value pairs.

Think of it as:

> "Take one key-value pair at a time, modify it if required, and return a new Object."

When do we use it?

You'll use `mapObject()` when:

- Renaming keys in an Object.
- Modifying values.
- Building dynamic payloads.
- Transforming API request or response Objects.

Syntax

```
payload mapObject (value, key, index) -> ...
```

The three parameters are:

- value → Current value
- key → Current key name
- index → Position of the key-value pair

You can use only the parameters that you need.

Important Points

- `mapObject()` works only with Objects.
- It always returns an Object.
- It is commonly used for dynamic Object transformations.
- Do not confuse it with `map()`.

Easy (Level 1)

Question

Convert all values to uppercase.

Input

```
{
    "firstName": "john",
    "lastName": "doe"
}
```

Code

```
%dw 2.0
output application/json
---
payload mapObject (value, key) -> {
    (key): upper(value)
}
```

Output

```
{
    "firstName": "JOHN",
    "lastName": "DOE"
}
```

Medium (Level 2)

Question

Rename the keys of an Object.

Input

```
{
    "employeeName": "Emma Wilson",
```

```
    "designation": "Integration Developer"
}
```

Code

```
%dw 2.0
output application/json
---
payload mapObject (value, key) -> {
    ("employee_" ++ key): value
}
```

Output

```
{
    "employee_employeeName": "Emma Wilson",
    "employee_designation": "Integration Developer"
}
```

Difficult (Level 3) - Real-Time Scenario

Question

An API receives customer information and the downstream system expects all field names to be prefixed with `customer_`.

Input

```
{
    "customerId": "CUST-1001",
    "customerName": "Alice Johnson",
    "membershipType": "Premium"
}
```

Code

```
%dw 2.0
output application/json
---
payload mapObject (value, key) -> {
```

```
    ("customer_" ++ key): value
}
```

Output

```
{
    "customer_customerId": "CUST-1001",
    "customer_customerName": "Alice Johnson",
    "customer_membershipType": "Premium"
}
```

Common Mistake

Many beginners write:

```
payload map ...
```

for an Object payload.

Remember:

```
Array   -> map()

Object  -> mapObject()
```

Choosing the correct function is extremely important in DataWeave.

Interview Tip

A very common MuleSoft interview question is:

> What is the difference between `map()` and `mapObject()`?

Answer:

> `map()` is used to transform Arrays, whereas `mapObject()` is used to transform Objects.

Pro Tip

Before writing any DataWeave transformation, identify the payload type.

Ask yourself:

- Is it an Array?
- Is it an Object?
- Is it an Array of Objects?

Once you know the answer, selecting the correct DataWeave function becomes much easier.

Topic 16 - pluck()

Time to Read: 4 Minutes

What does it do?

`pluck()` is used to extract values from an Object and return them as an Array.

Think of it as:

> "Take the values from an Object and put them into an Array."

It can also access:

- Value
- Key
- Index

When do we use it?

You'll use `pluck()` when:

- Converting Objects into Arrays.
- Extracting values from API responses.
- Preparing data for further transformations.
- Working with dynamic Object structures.

Syntax

```dw id="m8o5a2" payload pluck (value, key, index) -> ...

```
The parameters are:

- value → Current value
- key → Current key name
- index → Position of the key-value pair

Important Points
```
```

````
- `pluck()` works only with Objects.
- It always returns an Array.
- It is useful when an API returns data as an Object but another system expects
an Array.
- It is commonly used along with `map()` and `filter()`.

---

## Easy (Level 1)

### Question

Return all values from an Object.

### Input

```json id="1qz7m0"
{
    "city1": "Delhi",
    "city2": "Mumbai",
    "city3": "Pune"
}
````

Code

```dw id="o8d1vk" %dw 2.0 output application/json

---

payload pluck $

````
Output

```json id="wt8uaf"
[
    "Delhi",
    "Mumbai",
    "Pune"
]
````

````

Medium (Level 2)

Question

Return only the keys as an Array.

Input

```json id="nbaj8v"
{
  "name": "John Doe",
  "country": "India",
  "role": "Developer"
}
```

````
### Code

```dw id="x64i9u"
%dw 2.0
output application/json
---
payload pluck ((value, key) -> key)
```
````

Output

```json id="k1ckr7"
[
  "name",
  "country",
  "role"
]
```

````
---

## Difficult (Level 3) - Real-Time Scenario

### Question

An API returns customer information as an Object. The downstream system expects
an Array of customer details.

### Input

```json id="j2zw4u"
{
    "customerId": "CUST-1001",
    "customerName": "Emma Wilson",
    "membershipType": "Premium"
}
```
````

Code

```dw id="1uvl8y"
%dw 2.0
output application/json
```

---

payload pluck (value, key) -> { fieldName: key, fieldValue: value }

```
### Output

```json id="98lwqf"
[
    {
        "fieldName": "customerId",
        "fieldValue": "CUST-1001"
    },
    {
        "fieldName": "customerName",
        "fieldValue": "Emma Wilson"
    },
    {
        "fieldName": "membershipType",
        "fieldValue": "Premium"
    }
]
```

Common Mistake

Many developers confuse `pluck()` with `mapObject()`.

Remember:

```
mapObject() -> Object to Object

pluck()     -> Object to Array
```

The output type is the biggest difference between the two functions.

Interview Tip

A common interview question is:

> What is the difference between `pluck()` and `mapObject()`?

Answer:
```

`pluck()` converts an Object into an Array, whereas `mapObject()` transforms an Object and returns another Object.

Pro Tip

If an API returns dynamic Object fields and your downstream system expects an Array, `pluck()` is usually one of the cleanest solutions available in DataWeave.

Topic 17 - distinctBy()

Time to Read: 4 Minutes

What does it do?

`distinctBy()` is used to remove duplicate records from an Array.

Think of it as:

> "Keep only the unique records."

It is extremely useful when APIs return duplicate data and your downstream system expects only unique values.

When do we use it?

You'll use `distinctBy()` when:

- Removing duplicate customers.
- Eliminating repeated products.
- Processing unique order records.
- Cleaning API responses.
- Preparing data before sending it to another system.

Syntax

```
payload distinctBy (condition)
```

Important Points

- `distinctBy()` works with Arrays.
- It removes duplicate values based on the condition provided.
- It returns only unique records.
- It is frequently used along with `map()` and `filter()`.

Easy (Level 1)

Question

Remove duplicate numbers.

Input

```
[
    10,
    20,
    10,
    30,
    20
]
```

Code

```
%dw 2.0
output application/json
---
payload distinctBy $
```

Output

```
[
    10,
    20,
    30
]
```

Medium (Level 2)

Question

Remove duplicate employee names.

Input

```
[
    "John Doe",
```

```
    "Emma Wilson",
    "John Doe",
    "Michael Smith"
]
```

Code

```
%dw 2.0
output application/json
---
payload distinctBy $
```

Output

```
[
    "John Doe",
    "Emma Wilson",
    "Michael Smith"
]
```

Difficult (Level 3) - Real-Time Scenario

Question

A Customer API returns duplicate customer records. Return only unique customers based on the customer ID.

Input

```
[
    {
        "customerId": "CUST-1001",
        "customerName": "Alice Johnson"
    },
    {
        "customerId": "CUST-1002",
        "customerName": "Emma Wilson"
    },
    {
        "customerId": "CUST-1001",
        "customerName": "Alice Johnson"
```

```
    }
]
```

Code

```
%dw 2.0
output application/json
---
payload distinctBy $.customerId
```

Output

```
[
    {
        "customerId": "CUST-1001",
        "customerName": "Alice Johnson"
    },
    {
        "customerId": "CUST-1002",
        "customerName": "Emma Wilson"
    }
]
```

Common Mistake

Many developers use `filter()` when they actually need `distinctBy()`.

Remember:

```
filter()      -> Removes unwanted records.

distinctBy() -> Removes duplicate records.
```

Both solve completely different problems.

Interview Tip

A common MuleSoft interview question is:

> Which DataWeave function would you use to remove duplicate customer records?

Answer:

`distinctBy()` is the preferred function for removing duplicate records based on a specific field or value.

Pro Tip

A very common real-world pattern is:

```
filter()
↓
distinctBy()
↓
map()
```

For example:

1. Filter only active customers.
2. Remove duplicate customer IDs.
3. Transform the payload into the required API format.

This pattern is frequently used in enterprise integrations.

Topic 18 - orderBy()

Time to Read: 4 Minutes

What does it do?

`orderBy()` is used to sort records in ascending order.

Think of it as:

> "Arrange the data in a particular order."

Sorting is a very common requirement when working with API responses and reports.

When do we use it?

You'll use `orderBy()` when:

- Sorting customer names.
- Sorting employee salaries.
- Sorting order amounts.
- Sorting products by price.
- Preparing API responses in a readable format.

Syntax

```dw id="g4t1np" payload orderBy (condition)

```

Important Points

- `orderBy()` works with Arrays.
- By default, it sorts the records in ascending order.
- It can sort Strings, Numbers, and Object fields.
- It is commonly used before returning API responses.

Easy (Level 1)
```
```

```
### Question

Sort the numbers in ascending order.

### Input

```json id="p7o5vx"
[
    40,
    10,
    30,
    20
]
```
```

Code

```dw id="4ng3yz" %dw 2.0 output application/json

---

payload orderBy $

```
Output

```json id="mjef7z"
[
    10,
    20,
    30,
    40
]
```

Medium (Level 2)

Question

Sort employees based on their salary.

Input

```json id="dfg02u" [ { "employeeName": "John Doe", "salary": 85000 }, { "employeeName": "Emma Wilson", "salary": 65000 }, { "employeeName": "Michael Smith", "salary": 95000 } ]


```

```
### Code

```dw id="6q4n6w"
%dw 2.0
output application/json
---
payload orderBy $.salary
```

Output

```json id="w3m5lc" [ { "employeeName": "Emma Wilson", "salary": 65000 }, { "employeeName": "John Doe", "salary": 85000 }, { "employeeName": "Michael Smith", "salary": 95000 } ]
```

```
---

## Difficult (Level 3) - Real-Time Scenario

### Question

A Customer API returns the following records. Your requirements are:

- Return only Premium customers.
- Remove duplicate customer IDs.
- Sort customers by customer name.
- Return only the required fields.

### Input

```json id="sjg4jw"
[
    {
        "customerId": "CUST-1002",
        "customerName": "Emma Wilson",
        "membershipType": "Premium"
    },
    {
        "customerId": "CUST-1001",
        "customerName": "Alice Johnson",
        "membershipType": "Premium"
    },
    {
        "customerId": "CUST-1002",
        "customerName": "Emma Wilson",
        "membershipType": "Premium"
    },
```

```
    {
        "customerId": "CUST-1003",
        "customerName": "David Miller",
        "membershipType": "Standard"
    }
]
```

Code

```dw id="sk4o2z" %dw 2.0 output application/json

---

payload filter ($.membershipType == "Premium") distinctBy $.customerId orderBy $.customerName map { customerId: $.customerId, customerName: $.customerName }

### Output

```json id="g7a97s"
[
    {
        "customerId": "CUST-1001",
        "customerName": "Alice Johnson"
    },
    {
        "customerId": "CUST-1002",
        "customerName": "Emma Wilson"
    }
]
```

Common Mistake

Many developers assume that `orderBy()` sorts data in descending order.

Remember:

- `orderBy()` sorts records in ascending order by default.

If descending order is required, additional logic can be applied after sorting.

```

Interview Tip

A common interview question is:

> Can `orderBy()` be used on Arrays of Objects?

Answer:

> Yes. `orderBy()` is commonly used to sort Arrays of Objects based on fields such as salary, customer name, order amount, or employee ID.

Pro Tip

Sorting is usually the final step before returning an API response. In real-world MuleSoft projects, `orderBy()` is often combined with:

- `filter()`
- `distinctBy()`
- `map()`

This combination helps produce clean, unique, and properly ordered API responses.

Topic 19 - groupBy()

Time to Read: 5 Minutes

What does it do?

`groupBy()` is used to group records based on a common field or condition.

Think of it as:

> "Put similar records into the same bucket."

This function is extremely useful when you want to categorize or organize your data.

When do we use it?

You'll use `groupBy()` when:

- Grouping employees by department.
- Grouping orders by payment status.
- Grouping customers by membership type.
- Preparing summary reports.
- Building categorized API responses.

Syntax

```dw id="n4s9vb" payload groupBy (condition)
```

```
---

## Important Points

- `groupBy()` works with Arrays.
- It returns an Object, NOT an Array.
- Records with the same value are grouped together.
- This is one of the most commonly misunderstood interview topics.

---

## Easy (Level 1)
```

```
### Question

Group numbers based on whether they are even or odd.

### Input

```json id="g5lv8j"
[
    10,
    15,
    20,
    25
]
```

Code

```dw id="up9j4g" %dw 2.0 output application/json

---

payload groupBy ( if ($ mod 2 == 0) "Even Numbers" else "Odd Numbers" )

```
Output

```json id="k4bo5t"
{
    "Even Numbers": [
        10,
        20
    ],
    "Odd Numbers": [
        15,
        25
    ]
}
```

Medium (Level 2)

Question

Group employees by department.
```

Input

```json id="r83mn7" [ { "employeeName": "John Doe", "department": "IT" }, { "employeeName": "Emma Wilson", "department": "HR" }, { "employeeName": "Michael Smith", "department": "IT" } ]
```

````
### Code

```dw id="sz7y1m"
%dw 2.0
output application/json
---
payload groupBy $.department
```
````

Output

```json id="8xt7my" { "IT": [ { "employeeName": "John Doe", "department": "IT" }, { "employeeName": "Michael Smith", "department": "IT" } ], "HR": [ { "employeeName": "Emma Wilson", "department": "HR" } ] }
```

````
---

## Difficult (Level 3) - Real-Time Scenario

### Question

An Order API returns multiple orders. Group the successful and failed orders separately.

### Input

```json id="1z4g2n"
[
    {
        "orderId": "ORD-1001",
        "paymentStatus": "Success"
    },
    {
        "orderId": "ORD-1002",
        "paymentStatus": "Failed"
    },
    {
        "orderId": "ORD-1003",
        "paymentStatus": "Success"
    }
]
```
````

Code

```dw id="q8t5zr" %dw 2.0 output application/json

---

payload groupBy $.paymentStatus

### Output

```json id="x3mc1h"
{
    "Success": [
        {
            "orderId": "ORD-1001",
            "paymentStatus": "Success"
        },
        {
            "orderId": "ORD-1003",
            "paymentStatus": "Success"
        }
    ],
    "Failed": [
        {
            "orderId": "ORD-1002",
            "paymentStatus": "Failed"
        }
    ]
}
```

Common Mistake

This is probably the biggest mistake developers make:

> Many people think `groupBy()` returns an Array.

It does NOT.

Remember:

```text id="s92fh7" map() -> Array

filter() -> Array

orderBy() -> Array

distinctBy()-> Array

groupBy() -> Object ```

This is a very popular MuleSoft interview question.

---

## Interview Tip

A commonly asked interview question is:

> What is the return type of `groupBy()`?

**Answer:**

> `groupBy()` returns an Object where each key represents the grouping condition and the corresponding value contains the grouped records.

---

## Pro Tip

Whenever you need to create reports, categorized responses, or grouped API payloads, `groupBy()` is usually the cleanest and most readable solution available in DataWeave.

# Topic 20 - flatten()

**Time to Read:** 3 Minutes

## What does it do?

`flatten()` is used to convert nested Arrays into a single Array.

Think of it as:

> "Remove one level of nesting from an Array."

This function is extremely useful when APIs or transformations return Arrays inside Arrays.

---

## When do we use it?

You'll use `flatten()` when:

- Processing nested API responses.
- Combining records from multiple sources.
- Working with Scatter-Gather responses.
- Handling complex DataWeave transformations.
- Preparing data for downstream systems.

---

## Syntax

```
flatten(array)
```

Important Points

- `flatten()` works only with Arrays.
- It removes one level of nesting.
- It returns a single Array.
- It is commonly used after `map()` and other transformations.

Easy (Level 1)

Question

Convert a nested Array into a single Array.

Input

```
[
    [10, 20],
    [30, 40]
]
```

Code

```
%dw 2.0
output application/json
---
flatten(payload)
```

Output

```
[
    10,
    20,
    30,
    40
]
```

Medium (Level 2)

Question

Flatten a nested Array of product names.

Input

```
[
    ["Laptop", "Keyboard"],
```

```
    ["Mouse", "Monitor"]
]
```

Code

```
%dw 2.0
output application/json
---
flatten(payload)
```

Output

```
[
    "Laptop",
    "Keyboard",
    "Mouse",
    "Monitor"
]
```

Difficult (Level 3) - Real-Time Scenario

Question

A MuleSoft application receives customer records from two different APIs. The downstream system expects all customers in a single Array.

Input

```
[
    [
        {
            "customerId": "CUST-1001",
            "customerName": "Alice Johnson"
        }
    ],
    [
        {
            "customerId": "CUST-1002",
            "customerName": "Emma Wilson"
        }
```

```
    ]
]
```

Code

```
%dw 2.0
output application/json
---
flatten(payload)
```

Output

```
[
    {
        "customerId": "CUST-1001",
        "customerName": "Alice Johnson"
    },
    {
        "customerId": "CUST-1002",
        "customerName": "Emma Wilson"
    }
]
```

Common Mistake

Many developers expect `flatten()` to remove all levels of nesting.

For example:

Input

```
[
    [
        [10, 20]
    ]
]
```

Output

```
[
    [10, 20]
]
```

Notice that one level of nesting still exists.

Remember:

> `flatten()` removes only one level of nesting at a time.

Interview Tip

A common interview question is:

> When would you use `flatten()` in MuleSoft?

Answer:

> `flatten()` is commonly used when APIs or Mule components return nested Arrays and a downstream system expects the data in a single Array.

Pro Tip

If you're working with Scatter-Gather in MuleSoft, you'll frequently receive nested Arrays from multiple routes. `flatten()` is often one of the simplest ways to prepare the final payload before sending it to another system.

Topic 21 - reduce()

Time to Read: 6 Minutes

What does it do?

`reduce()` is used to combine multiple values into a single result.

Think of it as:

> "Take many values and produce one final value."

Unlike `map()` and `filter()`, which return Arrays, `reduce()` returns a single result.

That result can be:

- A Number
- A String
- An Object
- An Array
- Any valid DataWeave value

When do we use it?

You'll use `reduce()` when:

- Calculating total order amounts.
- Finding total sales.
- Combining multiple values.
- Building summary reports.
- Performing aggregations.

Syntax

```dw id="e4fknr" payload reduce (item, accumulator) -> ...
```

```
Where:

- `item` = Current value being processed.
- `accumulator` = Stores the result at every iteration.
```

````
Think of the accumulator as:

> "The running total."

---

## Important Points

- `reduce()` works with Arrays.
- It returns a single value.
- It is commonly used for calculations and aggregations.
- It is one of the most frequently asked DataWeave interview topics.

---

## Easy (Level 1)

### Question

Calculate the sum of all numbers.

### Input

```json id="mf68nq"
[
    10,
    20,
    30,
    40
]
````

Code

```dw id="e2z7mx"
%dw 2.0
output application/json
```

---

payload reduce ((item, accumulator) -> item + accumulator)

````
### Output

```json id="vngd4z"
100
````

Medium (Level 2)

Question

Calculate the total salary of all employees.

Input

```json id="dsmk27" [ { "salary": 80000 }, { "salary": 90000 }, { "salary": 70000 } ]
```

```
### Code

```dw id="vyq9my"
%dw 2.0
output application/json
---
payload reduce ((item, accumulator) ->
    item.salary + accumulator
)
```

Output

```json id="v9mn6v" 240000
```

```
---

## Difficult (Level 3) - Real-Time Scenario

### Question

An Order API returns multiple orders. Calculate the total revenue generated from
all successful orders.

### Input

```json id="c64vgf"
[
    {
        "orderAmount": 12000,
        "paymentStatus": "Success"
    },
    {
        "orderAmount": 5000,
        "paymentStatus": "Failed"
    },
```

```
    {
        "orderAmount": 8000,
        "paymentStatus": "Success"
    }
]
```

Code

```dw id="px4kgd" %dw 2.0 output application/json

---

payload filter ($.paymentStatus == "Success") reduce ((item, accumulator) -> item.orderAmount + accumulator )

### Output

```json id="6mwm53"
20000
```

Visual Representation

```text id="njh1iz" [10,20,30]

↓

10 + 20 + 30

↓

60
```

Another example:

```text id="1rb0ct"
All Orders

↓

filter()

↓
```
```

```
Successful Orders

↓

reduce()

↓

Total Revenue
```

This is exactly how many real-world MuleSoft transformations are written.

Common Mistake

Many developers confuse `map()` and `reduce()`.

Remember:

```text id="jw0spn"
map() ↓

Array in ↓

Array out

reduce() ↓

Array in ↓

Single value out
```

This difference is extremely important for interviews.

Interview Tip

A very common MuleSoft interview question is:

> When would you use `reduce()` instead of `map()`?

Answer:

Use `reduce()` when you need a single aggregated result such as total sales, total salary, total revenue, or summary values. Use `map()` when you need to transform every item in an Array and return another Array.

Pro Tip

Whenever you hear business requirements like:

- Total
- Sum
- Revenue
- Count
- Aggregate
- Summary

Immediately think of `reduce()`.

It is often the cleanest solution for aggregation-related transformations.

Topic 22 - contains()

Time to Read: 3 Minutes

What does it do?

`contains()` is used to check whether a value exists inside a String or an Array.

Think of it as:

> "Does this value exist?"

If the value exists, it returns:

- `true`

Otherwise, it returns:

- `false`

When do we use it?

You'll use `contains()` when:

- Validating customer roles.
- Checking supported countries.
- Validating API request values.
- Searching for keywords in Strings.
- Applying business rules.

Syntax

```
contains(value, searchValue)
```

or

```
payload contains "value"
```

Important Points

- `contains()` returns a Boolean value.
- It works with both Strings and Arrays.
- It is commonly used inside `if-else` conditions.
- It is very useful for simple validations.

Easy (Level 1)

Question

Check whether a String contains the word "MuleSoft".

Input

```
"MuleSoft Developer"
```

Code

```
%dw 2.0
output application/json
---
payload contains "MuleSoft"
```

Output

```
true
```

Medium (Level 2)

Question

Check whether an Array contains the value "India".

Input

```
[
    "India",
    "USA",
```

```
    "UK"
]
```

Code

```
%dw 2.0
output application/json
---
payload contains "India"
```

Output

```
true
```

Difficult (Level 3) - Real-Time Scenario

Question

An API receives customer information. Premium support is available only for customers from India and Singapore.

Input

```
{
    "customerName": "Emma Wilson",
    "country": "India"
}
```

Code

```
%dw 2.0
output application/json
var supportedCountries = ["India", "Singapore"]
---
{
    customerName: payload.customerName,
    premiumSupport:
        if (supportedCountries contains payload.country)
            true
        else
```

```
            false
}
```

Output

```
{
    "customerName": "Emma Wilson",
    "premiumSupport": true
}
```

Common Mistake

Many beginners assume that `contains()` performs partial matching in Arrays.

Example:

```
[
    "Premium Customer"
]
```

The following will return:

```
payload contains "Premium"
```

Output:

```
false
```

Remember:

> For Arrays, `contains()` checks for exact values.

Interview Tip

A common interview question is:

> What is the return type of `contains()`?

Answer:

`contains()` always returns a Boolean value (`true` or `false`).

Pro Tip

Whenever you're validating values such as:

- Countries
- Roles
- Membership types
- Supported payment methods
- Allowed API operations

`contains()` is usually one of the cleanest and most readable solutions in DataWeave.

Topic 23 - sizeOf()

Time to Read: 3 Minutes

What does it do?

`sizeOf()` is used to determine the size or length of data.

Think of it as:

> "How many items or characters do I have?"

It is commonly used for validations and business rules.

When do we use it?

You'll use `sizeOf()` when:

- Counting records returned by an API.
- Checking whether an Array contains data.
- Validating input values.
- Applying business rules based on the number of records.
- Building summary responses.

Syntax

```dw id="6wzkm5" sizeOf(value)
```

```
---

## Important Points

- `sizeOf()` works with Arrays and Strings.
- It returns a Number.
- It is commonly used for validations.
- It is frequently used inside `if-else` statements.

---

## Easy (Level 1)
```

```
### Question

Find the number of characters in a String.

### Input

```json id="e0kwm7"
"MuleSoft"
```

Code

```dw id="2w5t1x"
%dw 2.0
output application/json
---
sizeOf(payload)
```

Output

```
8
```

Medium (Level 2)

Question

Find the number of products in an Array.

Input

```json id="7owll5"
[ "Laptop", "Keyboard", "Mouse" ]
```

Code

```dw id="aljlwm"
%dw 2.0
output application/json
---
sizeOf(payload)
```

Output

```json id="vyrshs"
3
```
```

```
---

## Difficult (Level 3) - Real-Time Scenario

### Question

An API returns customer records. If more than 5 customers are returned, display
a special message.

### Input

```json id="fxl11g"
[
    {
        "customerId": "CUST-1001"
    },
    {
        "customerId": "CUST-1002"
    },
    {
        "customerId": "CUST-1003"
    },
    {
        "customerId": "CUST-1004"
    },
    {
        "customerId": "CUST-1005"
    },
    {
        "customerId": "CUST-1006"
    }
]
```

Code

```dw id="yz5f6l" %dw 2.0 output application/json

---

{ totalCustomers: sizeOf(payload), message: if (sizeOf(payload) > 5) "Large Customer Dataset" else "Small Customer Dataset" }

```
Output

```json id="2llx1v"
{
```
```

```
    "totalCustomers": 6,
    "message": "Large Customer Dataset"
}
```

Common Mistake

Many developers use `sizeOf()` multiple times in the same transformation.

Instead of:

```dw id="ld1glh" sizeOf(payload)

sizeOf(payload)

sizeOf(payload) ```

Consider storing the result in a variable if it is used repeatedly.

This improves readability and performance.

---

## Interview Tip

A common MuleSoft interview question is:

> What is the return type of `sizeOf()`?

**Answer:**

> `sizeOf()` always returns a Number representing the size or length of the supplied value.

---

## Pro Tip

Whenever business requirements mention:

- Total records
- Total customers
- Total orders
- Number of products
- Character count
```

Think of `sizeOf()` immediately. It is one of the simplest ways to perform such validations and calculations.

Topic 24 - isEmpty()

Time to Read: 3 Minutes

What does it do?

`isEmpty()` is used to check whether a value is empty.

Think of it as:

> "Do I have any data to process?"

It returns:

- `true` if the value is empty.
- `false` if the value contains data.

When do we use it?

You'll use `isEmpty()` when:

- Validating API responses.
- Checking whether an Array is empty.
- Handling optional values.
- Preventing transformation errors.
- Returning meaningful error or success messages.

Syntax

```
isEmpty(value)
```

Important Points

- `isEmpty()` returns a Boolean value.
- It is commonly used with Arrays and Strings.
- It helps prevent unexpected API failures.
- It is frequently used inside `if-else` statements.

Easy (Level 1)

Question

Check whether an Array is empty.

Input

```
[]
```

Code

```
%dw 2.0
output application/json
---
isEmpty(payload)
```

Output

```
true
```

Medium (Level 2)

Question

Check whether a customer name is empty.

Input

```
""
```

Code

```
%dw 2.0
output application/json
---
isEmpty(payload)
```

Output

```
true
```

Difficult (Level 3) - Real-Time Scenario

Question

An API returns a list of customer records. If no customers are found, return a friendly message.

Input

```
[]
```

Code

```
%dw 2.0
output application/json
---
if (isEmpty(payload))
{
    message: "No customer records found."
}
else
{
    message: "Customer records are available.",
    totalCustomers: sizeOf(payload)
}
```

Output

```
{
    "message": "No customer records found."
}
```

Common Mistake

Many developers write:

```
sizeOf(payload) == 0
```

While this works, the following is much cleaner and easier to read:

```
isEmpty(payload)
```

Whenever you're simply checking whether data exists, prefer using `isEmpty()`.

Interview Tip

A common MuleSoft interview question is:

> Why do we use `isEmpty()` in DataWeave?

Answer:

> `isEmpty()` is used to validate whether incoming data is empty before processing it. It helps avoid unnecessary transformations and improves error handling in MuleSoft applications.

Pro Tip

Always validate API responses before processing them.

A simple check like:

```
if (isEmpty(payload))
```

can prevent many runtime issues in production environments.

Topic 25 - default Keyword

Time to Read: 4 Minutes

What does it do?

The `default` keyword is used to provide a fallback value when the actual value is null or missing.

Think of it as:

> "If the value is not available, use this value instead."

When do we use it?

You'll use `default` when:

- API fields are missing.
- Customer information is incomplete.
- Optional fields are not present.
- Handling null values.
- Providing meaningful default responses.

Syntax

```
value default "Some Default Value"
```

Important Points

- `default` is used for null or missing values.
- It improves API reliability.
- It makes transformations cleaner and easier to read.
- It is one of the most commonly used DataWeave features in MuleSoft projects.

Easy (Level 1)

Question

If the customer's city is missing, return "Not Available".

Input

```
{
    "customerName": "John Doe"
}
```

Code

```
%dw 2.0
output application/json
---
{
    city: payload.city default "Not Available"
}
```

Output

```
{
    "city": "Not Available"
}
```

Medium (Level 2)

Question

Provide default values for employee details.

Input

```
{
    "employeeName": "Emma Wilson"
}
```

Code

```
%dw 2.0
output application/json
---
{
    employeeName: payload.employeeName,
    designation: payload.designation default "Associate Developer",
    country: payload.country default "India"
}
```

Output

```
{
    "employeeName": "Emma Wilson",
    "designation": "Associate Developer",
    "country": "India"
}
```

Difficult (Level 3) - Real-Time Scenario

Question

A Customer API sometimes sends incomplete information. The downstream system expects all fields to be present.

Input

```
{
    "customerId": "CUST-1001",
    "customerName": "Alice Johnson"
}
```

Code

```
%dw 2.0
output application/json
---
{
    customerId: payload.customerId,
    customerName: payload.customerName,
```

```
    membershipType: payload.membershipType default "Standard",
    country: payload.country default "India",
    rewardPoints: payload.rewardPoints default 0,
    emailVerified: payload.emailVerified default false
}
```

Output

```
{
    "customerId": "CUST-1001",
    "customerName": "Alice Johnson",
    "membershipType": "Standard",
    "country": "India",
    "rewardPoints": 0,
    "emailVerified": false
}
```

Common Mistake

Many developers write complex `if-else` statements for null checks.

Instead of:

```
if (payload.country != null)
    payload.country
else
    "India"
```

Simply write:

```
payload.country default "India"
```

The code is cleaner and much easier to maintain.

Interview Tip

A very common MuleSoft interview question is:

> Why do we use the `default` keyword in DataWeave?

Answer:

> The `default` keyword is used to provide fallback values when fields are null or missing, making API transformations more robust and easier to maintain.

Pro Tip

Whenever you're working with external APIs, assume that some fields might be missing. Using the `default` keyword proactively can prevent many transformation issues in production environments.

Topic 26 - String Functions (upper(), lower(), trim(), capitalize())

Time to Read: 5 Minutes

What do they do?

These functions help you format and clean String values.

Function	Purpose
upper()	Converts text to uppercase
lower()	Converts text to lowercase
trim()	Removes leading and trailing spaces
capitalize()	Capitalizes the first letter of a word

Think of them as:

> "Clean and format your String data before sending it to another system."

When do we use them?

You'll use these functions when:

- Formatting customer names.
- Cleaning API request payloads.
- Standardizing user inputs.
- Building consistent API responses.
- Processing data received from external systems.

Important Points

- These functions work with Strings.
- They are lightweight and easy to use.
- They are commonly used with `map()` and `default`.
- They improve data consistency across integrations.

Easy (Level 1)

Question

Convert a customer's name to uppercase.

Input

```
{
    "customerName": "john doe"
}
```

Code

```
%dw 2.0
output application/json
---
{
    customerName: upper(payload.customerName)
}
```

Output

```
{
    "customerName": "JOHN DOE"
}
```

Medium (Level 2)

Question

Clean and format the customer's name.

Input

```
{
    "customerName": "   EMMA WILSON   "
}
```

Code

```
%dw 2.0
output application/json
---
{
    customerName: lower(trim(payload.customerName))
}
```

Output

```
{
    "customerName": "emma wilson"
}
```

Difficult (Level 3) - Real-Time Scenario

Question

A Customer API receives inconsistent customer names. Your requirements are:

- Remove unwanted spaces.
- Convert the name to lowercase.
- Capitalize the first letter.
- Return the cleaned customer name.

Input

```
{
    "customerName": "   ALICE JOHNSON   "
}
```

Code

```
%dw 2.0
output application/json
---
{
    customerName:
        capitalize(
            lower(
                trim(payload.customerName)
```

```
            )
        )
}
```

Output

```
{
    "customerName": "Alice johnson"
}
```

Common Mistake

Many developers assume that `capitalize()` converts every word into title case.

Incorrect assumption:

```
JOHN DOE

↓

John Doe
```

Actual output:

```
John doe
```

Remember:

> `capitalize()` only capitalizes the first character of the String.

Interview Tip

A common MuleSoft interview question is:

> Which String functions are most commonly used in DataWeave?

A practical answer is:

`upper()`, `lower()`, `trim()`, and `capitalize()` are some of the most frequently used String functions for formatting and cleaning data in MuleSoft projects.

Pro Tip

Whenever you're integrating with external systems, never assume the incoming data is properly formatted. Cleaning String values before processing them is considered a good practice in enterprise integrations.

Did You Know?

A very common real-world MuleSoft pattern is:

```
trim()
↓

default
↓

upper() or lower()

↓

map()

↓

Final API Response
```

This approach helps produce clean, consistent, and reliable API payloads.

Topic 27 - splitBy() and joinBy()

Time to Read: 4 Minutes

What do they do?

These two functions work in opposite directions.

Function	Purpose
splitBy()	Converts a String into an Array
joinBy()	Converts an Array into a String

Think of them as:

> "Split a String when you need individual values and join them when another system expects a single String."

When do we use them?

You'll use these functions when:

- Processing comma-separated values.
- Building API request payloads.
- Formatting email recipients.
- Handling product or skill lists.
- Preparing data for downstream systems.

Important Points

- `splitBy()` works on Strings.
- `joinBy()` works on Arrays.
- Both are commonly used in API transformations.
- They are simple but extremely useful in real projects.

Easy (Level 1)

Question

Convert a comma-separated String into an Array.

Input

```
"Java,MuleSoft,DataWeave"
```

Code

```
%dw 2.0
output application/json
---
payload splitBy ","
```

Output

```
[
    "Java",
    "MuleSoft",
    "DataWeave"
]
```

Medium (Level 2)

Question

Convert an Array of skills into a comma-separated String.

Input

```
[
    "Java",
    "MuleSoft",
    "DataWeave"
]
```

Code

```
%dw 2.0
output application/json
---
payload joinBy ", "
```

Output

```
"Java, MuleSoft, DataWeave"
```

Difficult (Level 3) - Real-Time Scenario

Question

A Customer API receives customer interests as a comma-separated String. Your requirements are:

- Split the values.
- Convert them into uppercase.
- Return them as an Array.

Input

```
{
    "interests": "books,travel,music"
}
```

Code

```
%dw 2.0
output application/json
---
{
    customerInterests:
        payload.interests
            splitBy ","
            map upper($)
}
```

Output

```
{
    "customerInterests": [
        "BOOKS",
        "TRAVEL",
        "MUSIC"
    ]
}
```

Common Mistake

Many developers confuse the two functions.

Remember:

```
String ---> splitBy() ---> Array

Array ---> joinBy() ---> String
```

If you're not sure which one to use, simply ask yourself:

> What is my input and what should my output look like?

Interview Tip

A common MuleSoft interview question is:

> When would you use `splitBy()` in DataWeave?

Answer:

> `splitBy()` is used when a String contains multiple values separated by a delimiter and we want to process them individually as an Array.

Pro Tip

A very common real-world pattern is:

```
splitBy()

↓

filter()

↓

map()

↓
```

```
joinBy()

↓

Final API Response
```

For example:

- Receive customer roles as a comma-separated String.
- Split the roles.
- Filter unwanted values.
- Format them.
- Join them back before sending them to another API.

This pattern is frequently used while integrating with legacy and external systems.

Topic 28 - replace()

Time to Read: 4 Minutes

What does it do?

`replace()` is used to replace one value or pattern with another value.

Think of it as:

> "Find this value and replace it with something else."

This function is extremely useful for cleaning and formatting API data before processing it.

When do we use it?

You'll use `replace()` when:

- Removing unwanted characters.
- Formatting customer names.
- Cleaning phone numbers.
- Updating String values.
- Standardizing incoming API payloads.

Syntax

```
replace(value, oldValue, newValue)
```

or

```
someString replace "oldValue" with "newValue"
```

Important Points

- `replace()` works with Strings.
- It is commonly used in API request and response transformations.
- It makes String manipulation simple and readable.

- It is frequently used with `trim()`, `upper()`, and `lower()`.

Easy (Level 1)

Question

Replace "Developer" with "Architect".

Input

```
"MuleSoft Developer"
```

Code

```
%dw 2.0
output application/json
---
payload replace "Developer" with "Architect"
```

Output

```
"MuleSoft Architect"
```

Medium (Level 2)

Question

Replace spaces with underscores.

Input

```
"Customer Support Team"
```

Code

```
%dw 2.0
output application/json
```

```
---
payload replace " " with "_"
```

Output

```
"Customer_Support_Team"
```

Difficult (Level 3) - Real-Time Scenario

Question

A Payment API sends phone numbers with spaces and hyphens. Your requirement is to clean the phone number before sending it to another system.

Input

```
{
    "phoneNumber": "98765-432 10"
}
```

Code

```
%dw 2.0
output application/json
---
{
    cleanedPhoneNumber:
        (payload.phoneNumber
            replace "-" with "")
            replace " " with ""
}
```

Output

```
{
    "cleanedPhoneNumber": "9876543210"
}
```

Common Mistake

Many beginners think `replace()` updates the original value.

Remember:

> DataWeave transformations are immutable.

`replace()` returns a new value. It does not modify the original payload.

Interview Tip

A common MuleSoft interview question is:

> Can we use multiple `replace()` operations in a single transformation?

Answer:

> Yes. Multiple `replace()` operations can be chained together to clean and format incoming data efficiently.

Pro Tip

Whenever you're integrating with external systems, expect inconsistent data formats. Functions like `trim()`, `replace()`, `upper()`, and `lower()` are often used together to normalize incoming String values before processing them.

Did You Know?

A very common MuleSoft pattern is:

```
trim()
↓

replace()
↓

lower()
↓

default
```

```
↓

Final API Response
```

This approach helps produce clean and predictable payloads, especially when consuming data from third-party APIs.

Topic 29 - Date and Time Functions

Time to Read: 5 Minutes

Why are Date and Time Functions Important?

Different systems use different date formats.

For example:

- 2026-07-16
- 16-07-2026
- 07/16/2026
- 2026-07-16T10:30:00Z

One of the most common MuleSoft requirements is:

> "Read the incoming date and convert it into the format expected by another system."

When do we use them?

You'll use Date and Time functions when:

- Formatting API request and response payloads.
- Transforming timestamps.
- Generating reports.
- Sending notifications.
- Integrating with Salesforce and external APIs.

Important Points

- DataWeave provides built-in support for Date and DateTime values.
- Date transformations are extremely common in enterprise integrations.
- Always verify the format expected by the downstream system.

Easy (Level 1)

Question

Return today's date.

Code

```
%dw 2.0
output application/json
---
now() as Date
```

Output

```
"2026-07-16"
```

> Note: The output depends on the date when the transformation is executed.

Medium (Level 2)

Question

Return the current date and time.

Code

```
%dw 2.0
output application/json
---
now()
```

Sample Output

```
"2026-07-16T10:30:45.123Z"
```

Difficult (Level 3) - Real-Time Scenario

Question

An Order API expects the order date in the following format:

```
dd-MM-yyyy
```

The incoming payload is:

```
{
    "orderDate": "2026-07-16"
}
```

Code

```
%dw 2.0
output application/json
---
{
    formattedOrderDate:
        (payload.orderDate as Date)
            as String {format: "dd-MM-yyyy"}
}
```

Output

```
{
    "formattedOrderDate": "16-07-2026"
}
```

Commonly Used Formats

Format	Example
yyyy-MM-dd	2026-07-16
dd-MM-yyyy	16-07-2026
MM/dd/yyyy	07/16/2026
yyyy/MM/dd	2026/07/16
HH:mm:ss	10:30:45

You don't need to memorize all formats. Just remember that DataWeave makes it easy to convert between them.

Common Mistake

Many developers assume that all APIs use the same date format.

Remember:

> Always validate the incoming and outgoing date formats.

A date transformation that works for one API may fail for another simply because of formatting differences.

Interview Tip

A common MuleSoft interview question is:

> How do you format a Date in DataWeave?

Answer:

> By converting it using `as String {format: "desired-format"}` after casting it to a Date or DateTime value.

Pro Tip

Whenever you're working with:

- Salesforce
- Payment systems
- ERP integrations
- Reporting APIs
- External vendors

there is a very high chance you'll need to perform date and time transformations.

Mastering date formatting early will make enterprise integrations much easier.

Did You Know?

A very common MuleSoft pattern is:

```
Receive Date
```

```
↓

Convert to Date

↓

Format as Required

↓

Build Final API Payload
```

Most production integrations don't fail because of business logic—they fail because of mismatched data formats. Date transformations are one of the most common examples of this.

Topic 30 - XML Transformations

Time to Read: 5 Minutes

Why is XML Important?

Although JSON is very popular, many enterprise systems still use XML.

You'll frequently encounter XML when working with:

- SOAP Web Services
- Legacy Systems
- ERP Integrations
- Banking Applications
- Healthcare Systems
- Third-party APIs

One of the most common MuleSoft requirements is:

> "Receive XML and convert it into JSON."

or

> "Convert JSON into XML before calling an external API."

When do We Use XML Transformations?

You'll use XML transformations when:

- Consuming SOAP APIs.
- Transforming legacy system responses.
- Integrating with enterprise applications.
- Building XML request payloads.
- Processing XML files.

Important Points

- DataWeave handles XML transformations very easily.
- XML fields are accessed just like JSON fields.
- XML to JSON and JSON to XML transformations are very common in MuleSoft projects.

Easy (Level 1)

Question

Convert an XML payload into JSON.

Input

```
<employee>
    <employeeId>101</employeeId>
    <employeeName>John Doe</employeeName>
</employee>
```

Code

```
%dw 2.0
output application/json
---
{
    employeeId: payload.employee.employeeId,
    employeeName: payload.employee.employeeName
}
```

Output

```
{
    "employeeId": "101",
    "employeeName": "John Doe"
}
```

Medium (Level 2)

Question

Convert a JSON payload into XML.

Input

```
{
    "customerId": "CUST-1001",
```

```
    "customerName": "Emma Wilson"
}
```

Code

```
%dw 2.0
output application/xml
---
customer: {
    customerId: payload.customerId,
    customerName: payload.customerName
}
```

Output

```
<customer>
    <customerId>CUST-1001</customerId>
    <customerName>Emma Wilson</customerName>
</customer>
```

Difficult (Level 3) - Real-Time Scenario

Question

A SOAP service returns customer information in XML. Your downstream REST API expects JSON.

Input

```
<customer>
    <customerId>CUST-1001</customerId>
    <customerName>Alice Johnson</customerName>
    <membershipType>Premium</membershipType>
</customer>
```

Code

```
%dw 2.0
output application/json
---
{
    customerId: payload.customer.customerId,
```

```
    customerName: payload.customer.customerName,
    membershipType: payload.customer.membershipType
}
```

Output

```
{
    "customerId": "CUST-1001",
    "customerName": "Alice Johnson",
    "membershipType": "Premium"
}
```

Common Mistake

Many beginners assume XML transformations are difficult.

Remember:

> DataWeave treats XML as structured data, making it very easy to read and transform.

Once the XML payload is parsed by MuleSoft, accessing the fields is straightforward.

Interview Tip

A common MuleSoft interview question is:

> Which transformation is more common in MuleSoft projects: JSON or XML?

Answer:

> JSON is more common in modern REST APIs, whereas XML is still widely used in enterprise and SOAP-based integrations. A MuleSoft developer should be comfortable working with both.

Pro Tip

If you're preparing for MuleSoft interviews, don't skip XML. Many experienced-level interviews include XML transformation questions because enterprise integrations frequently involve SOAP and legacy systems.

Did You Know?

A very common MuleSoft pattern is:

```
SOAP Service (XML)

↓

DataWeave Transformation

↓

JSON Payload

↓

REST API

↓

Final Response
```

This is one of the most common enterprise integration patterns you'll encounter as a MuleSoft developer.

Topic 31 - CSV Transformations

Time to Read: 5 Minutes

Why is CSV Important?

CSV (Comma-Separated Values) is one of the most widely used file formats in enterprise applications.

You'll frequently encounter CSV files when working with:

- Batch Processing
- Employee Records
- Product Catalogs
- Customer Data Imports
- File-based Integrations
- Legacy Systems

One of the most common MuleSoft requirements is:

> "Read a CSV file and transform it into JSON."

or

> "Convert API data into a CSV file for downstream systems."

When do We Use CSV Transformations?

You'll use CSV transformations when:

- Processing files from SFTP servers.
- Building batch integrations.
- Importing and exporting data.
- Generating reports.
- Integrating with legacy systems.

Important Points

- DataWeave supports CSV transformations out of the box.
- CSV can easily be converted into JSON and vice versa.
- CSV processing is commonly used in MuleSoft batch jobs.

Easy (Level 1)

Question

Convert CSV data into JSON.

Input

```
employeeId,employeeName
101,John Doe
102,Emma Wilson
```

Code

```
%dw 2.0
output application/json
---
payload
```

Output

```
[
    {
        "employeeId": "101",
        "employeeName": "John Doe"
    },
    {
        "employeeId": "102",
        "employeeName": "Emma Wilson"
    }
]
```

Medium (Level 2)

Question

Return only employee names from the CSV payload.

Input

```
employeeId,employeeName
101,John Doe
102,Emma Wilson
```

Code

```
%dw 2.0
output application/json
---
payload map {
    employeeName: $.employeeName
}
```

Output

```
[
    {
        "employeeName": "John Doe"
    },
    {
        "employeeName": "Emma Wilson"
    }
]
```

Difficult (Level 3) - Real-Time Scenario

Question

A CSV file contains customer information. Your requirements are:

- Return only Premium customers.
- Convert customer names to uppercase.
- Return only customer ID and customer name.

Input

```
customerId,customerName,membershipType
CUST-1001,Alice Johnson,Premium
```

```
CUST-1002,David Miller,Standard
CUST-1003,Emma Wilson,Premium
```

Code

```
%dw 2.0
output application/json
---
payload
    filter ($.membershipType == "Premium")
    map {
        customerId: $.customerId,
        customerName: upper($.customerName)
    }
```

Output

```
[
    {
        "customerId": "CUST-1001",
        "customerName": "ALICE JOHNSON"
    },
    {
        "customerId": "CUST-1003",
        "customerName": "EMMA WILSON"
    }
]
```

Common Mistake

Many beginners assume CSV data needs to be manually split and parsed.

Remember:

> MuleSoft automatically parses CSV data when configured correctly, allowing DataWeave to work with it just like an Array of Objects.

This means you can directly use:

- `map()`
- `filter()`
- `orderBy()`

- `distinctBy()`
- `reduce()`

on CSV payloads.

Interview Tip

A common MuleSoft interview question is:

> How does DataWeave represent CSV data internally?

Answer:

> DataWeave represents CSV records as an Array of Objects, making it easy to perform transformations using standard DataWeave functions.

Pro Tip

Whenever you're working with:

- Batch Jobs
- File Processing
- SFTP Integrations
- Large Data Imports

think of CSV transformations first. Most CSV processing requirements can be solved using the same DataWeave functions you've already learned for JSON payloads.

Did You Know?

A very common MuleSoft pattern is:

```
CSV File

↓

filter()

↓

map()
```

```
↓

orderBy()

↓

DataWeave Transformation

↓

JSON or CSV Output
```

Once you understand Arrays of Objects, CSV transformations become surprisingly easy because DataWeave treats CSV records just like JSON records.

Topic 32 - match Expression

Time to Read: 5 Minutes

What does it do?

The `match` expression is used to evaluate multiple conditions in a clean and readable way.

Think of it as:

> "Instead of writing many if-else statements, let DataWeave decide which condition matches."

It makes your transformations easier to read and maintain.

When do we use it?

You'll use `match` when:

- Handling customer membership types.
- Returning different API responses.
- Applying business rules.
- Performing value-based transformations.
- Simplifying complex conditional logic.

Syntax

```
value match {

    case condition1 -> output1

    case condition2 -> output2

    else -> defaultOutput

}
```

Important Points

- `match` improves readability.
- It is a great alternative to multiple if-else statements.
- It supports multiple cases and a default value.
- It is especially useful when business logic grows.

Easy (Level 1)

Question

Return the discount percentage based on membership type.

Input

```
{
    "membershipType": "Premium"
}
```

Code

```
%dw 2.0
output application/json
---
payload.membershipType match {

    case "Premium" -> "20%"

    case "Gold" -> "15%"

    else -> "5%"
}
```

Output

```
"20%"
```

Medium (Level 2)

Question

Return the payment status message.

Input

```
{
    "paymentStatus": "Success"
}
```

Code

```
%dw 2.0
output application/json
---
{
    message:
        payload.paymentStatus match {

            case "Success" -> "Payment Successful"

            case "Pending" -> "Payment is Pending"

            case "Failed" -> "Payment Failed"

            else -> "Unknown Payment Status"
        }
}
```

Output

```
{
    "message": "Payment Successful"
}
```

Difficult (Level 3) - Real-Time Scenario

Question

An Order API returns the customer's membership type. Your requirements are:

- Premium → Free Delivery + 20% Discount
- Gold → Priority Delivery + 15% Discount
- Standard → Standard Delivery + 5% Discount
- Any other value → Contact Support

Input

```
{
    "membershipType": "Gold"
}
```

Code

```
%dw 2.0
output application/json
---
payload.membershipType match {

    case "Premium" -> {
        deliveryType: "Free Delivery",
        discount: "20%"
    }

    case "Gold" -> {
        deliveryType: "Priority Delivery",
        discount: "15%"
    }

    case "Standard" -> {
        deliveryType: "Standard Delivery",
        discount: "5%"
    }

    else -> {
        message: "Please contact customer support."
    }

}
```

Output

```
{
    "deliveryType": "Priority Delivery",
    "discount": "15%"
}
```

Common Mistake

Many developers use nested if-else statements for multiple conditions.

Instead of:

```
if

↓

else if

↓

else if

↓

else if

↓

else
```

Prefer:

```
match

↓

case

↓

case
```

```
↓

case

↓

else
```

It is cleaner and much easier to maintain.

Interview Tip

A common MuleSoft interview question is:

> When should we use the match expression instead of if-else?

Answer:

> Use `match` when there are multiple conditions based on a particular value. It makes the transformation cleaner, more readable, and easier to maintain than nested if-else statements.

Pro Tip

Whenever business requirements include statements like:

- If Premium...
- If Gold...
- If Standard...
- Otherwise...

think of the `match` expression immediately.

It is one of the cleanest ways to implement value-based business rules in DataWeave.

Did You Know?

A very common real-world pattern is:

```
API Payload
```

```
↓

match Expression

↓

Business Rule Evaluation

↓

DataWeave Transformation

↓

Final API Response
```

This pattern is frequently used in enterprise MuleSoft integrations where different customer types, order statuses, or payment statuses require different responses.

Topic 33 - update Operator

Time to Read: 6 Minutes

What does it do?

The `update` operator is used to update or modify specific fields in an existing payload.

Think of it as:

> "Keep the original payload and update only the fields that need to change."

This makes your transformations:

- Cleaner
- More readable
- Easier to maintain

When do we use it?

You'll use the `update` operator when:

- Updating customer information.
- Modifying API responses.
- Changing field values.
- Applying business rules.
- Working with nested Objects.

Syntax

```dw id="1oh1lr" payload update {

```
case field at .fieldName -> newValue
```

}

```

Important Points
```
```

```
- The `update` operator was introduced in DataWeave 2.x.
- It updates only the specified fields.
- The remaining payload remains unchanged.
- It is very useful for large and complex payloads.

---

## Easy (Level 1)

### Question

Update the customer's country.

### Input

```json id="bsjlwm"
{
    "customerName": "John Doe",
    "country": "USA"
}
```

Code

```dw id="mq9hmu"
%dw 2.0
output application/json

---

payload update {

    case country at .country -> "India"

}
```

Output

```json id="owcvqv"
{
    "customerName": "John Doe",
    "country": "India"
}
```

```

Medium (Level 2)

Question

Update the membership type.

Input

```json id="v6n9st" { "customerId": "CUST-1001", "membershipType": "Standard" }

```
Code

```dw id="rz85e6"
%dw 2.0
output application/json
---
payload update {

    case membership at .membershipType -> "Premium"

}
```

Output

```json id="2d7o6e" { "customerId": "CUST-1001", "membershipType": "Premium" }

```

Difficult (Level 3) - Real-Time Scenario

Question

A Customer API returns the following payload. Your requirements are:

- Upgrade the membership type.
- Verify the email status.
- Keep all other fields unchanged.

Input

```json id="n4x3n1"
{
    "customerId": "CUST-1001",
    "customerName": "Alice Johnson",
    "membershipType": "Standard",
```

```
    "emailVerified": false,
    "country": "India"
}
```

Code

```dw id="q4u3ne" %dw 2.0 output application/json

---

payload update {

```
    case membership at .membershipType -> "Premium"

    case emailStatus at .emailVerified -> true

}
```

```

Output

```json id="o0a4ry"
{
    "customerId": "CUST-1001",
    "customerName": "Alice Johnson",
    "membershipType": "Premium",
    "emailVerified": true,
    "country": "India"
}
```

Why is it Better?

Without the `update` operator, many developers write:

```text id="kjlwmx" Copy the entire payload.

↓

Modify one or two fields.

↓

Return the new Object.
```

```
With the `update` operator:

```text id="gl6kn0"
Update only what is required.

↓

Everything else remains unchanged.
```

Much cleaner. Much easier to maintain.

Common Mistake

Many developers rebuild large Objects when only one field needs to be changed.

Instead of:

```text id="x5ty0k" 50 fields copied manually.

```
Simply use:

```text id="dxquka"
payload update { ... }
```

The transformation becomes significantly smaller and more readable.

Interview Tip

A common MuleSoft interview question is:

> Why would you use the update operator instead of rebuilding the payload?

Answer:

> The `update` operator makes transformations cleaner and easier to maintain by modifying only the required fields while keeping the remaining payload unchanged.

```

Pro Tip

If you're working with large API responses containing 30 or 40 fields, the `update` operator can save a lot of development time and significantly improve code readability.

Did You Know?

A very common real-world pattern is:

```text id="ln6r06" Receive API Payload

↓

Apply Business Rules

↓

Update Specific Fields

↓

Return Updated Payload ```

This pattern is extremely common when integrating with CRM systems, order management systems, and customer-facing APIs.
```

Topic 34 - Real-Time Scenario 1 (Customer API Transformation)

Time to Read: 6 Minutes

Problem Statement

A Customer API returns the following payload.

Your requirements are:

- Return only Active customers.
- Remove duplicate customer records.
- Convert customer names to uppercase.
- Sort customers by customer name.
- If membership type is missing, set it to "Standard".
- Return only the required fields.

This is a very realistic MuleSoft interview and project scenario.

Input

```
[
    {
        "customerId": "CUST-1003",
        "customerName": "Alice Johnson",
        "membershipType": "Premium",
        "status": "Active"
    },
    {
        "customerId": "CUST-1001",
        "customerName": "Emma Wilson",
        "status": "Active"
    },
    {
        "customerId": "CUST-1002",
        "customerName": "Michael Smith",
        "membershipType": "Gold",
        "status": "Inactive"
    },
    {
        "customerId": "CUST-1001",
```

```
        "customerName": "Emma Wilson",
        "status": "Active"
    }
]
```

Functions Used

- filter()
- distinctBy()
- orderBy()
- map()
- upper()
- default

Code

```
%dw 2.0
output application/json
---
payload
    filter ($.status == "Active")
    distinctBy $.customerId
    orderBy $.customerName
    map {
        customerId: $.customerId,
        customerName: upper($.customerName),
        membershipType: $.membershipType default "Standard"
    }
```

Output

```
[
    {
        "customerId": "CUST-1003",
        "customerName": "ALICE JOHNSON",
        "membershipType": "Premium"
    },
    {
        "customerId": "CUST-1001",
        "customerName": "EMMA WILSON",
```

```
        "membershipType": "Standard"
    }
]
```

What's Happening?

```
Input Payload

↓

filter()
(Return Active Customers)

↓

distinctBy()
(Removes Duplicate Customer IDs)

↓

orderBy()
(Sorts the Customers)

↓

map()
(Returns Required Fields)

↓

upper()
(Formats Customer Names)

↓

default
(Handles Missing Membership Types)

↓

Final API Response
```

Important Points

- Real-world transformations almost always combine multiple functions.
- Filtering should usually happen before mapping.
- Default values make APIs more reliable.
- Sorting is usually performed towards the end of the transformation.

Common Mistake

Many developers write multiple Transform Message components for simple transformations.

Instead of:

```
Transform Message 1

↓

Transform Message 2

↓

Transform Message 3
```

Prefer:

```
Single DataWeave Transformation

↓

Cleaner Code

↓

Better Performance

↓

Easier Maintenance
```

Whenever possible, keep related transformation logic together.

Interview Tip

A very common MuleSoft interview question is:

> Have you used multiple DataWeave functions together in real projects?

A good answer is:

> Yes. In real-world MuleSoft integrations, it is very common to combine functions such as filter(), map(), distinctBy(), orderBy(), default, and conditional logic to build API request and response payloads.

Pro Tip

Real MuleSoft projects rarely use only one DataWeave function at a time.

If you can comfortably combine:

- filter()
- map()
- default
- distinctBy()
- orderBy()
- if-else

you'll be able to solve a large percentage of day-to-day transformation requirements.

Topic 35 - Real-Time Scenario 2 (Order Processing API)

Time to Read: 6 Minutes

Problem Statement

An Order API returns the following payload.

Your requirements are:

- Return only successfully paid orders.
- Remove duplicate order IDs.
- Sort the orders based on the order amount.
- Calculate the total revenue.
- Return only the required fields.

This is a common requirement in e-commerce and payment integrations.

Input

```
[
    {
        "orderId": "ORD-1003",
        "orderAmount": 12000,
        "paymentStatus": "Success"
    },
    {
        "orderId": "ORD-1001",
        "orderAmount": 5000,
        "paymentStatus": "Success"
    },
    {
        "orderId": "ORD-1002",
        "orderAmount": 8000,
        "paymentStatus": "Failed"
    },
    {
        "orderId": "ORD-1001",
        "orderAmount": 5000,
        "paymentStatus": "Success"
```

```
    }
]
```

Functions Used

- filter()
- distinctBy()
- orderBy()
- map()
- reduce()

Code

```
%dw 2.0
output application/json

var successfulOrders =
    payload
        filter ($.paymentStatus == "Success")
        distinctBy $.orderId
        orderBy $.orderAmount

---
{
    totalRevenue:
        successfulOrders
            reduce ((item, accumulator) ->
                item.orderAmount + accumulator
            ),

    orders:
        successfulOrders map {
            orderId: $.orderId,
            orderAmount: $.orderAmount
        }
}
```

Output


```
{
    "totalRevenue": 17000,
    "orders": [
        {
            "orderId": "ORD-1001",
            "orderAmount": 5000
        },
        {
            "orderId": "ORD-1003",
            "orderAmount": 12000
        }
    ]
}
```

What's Happening?

```
Input Payload

↓

filter()
(Return Successful Orders)

↓

distinctBy()
(Removes Duplicate Orders)

↓

orderBy()
(Sorts the Orders)

↓

reduce()
(Calculates Total Revenue)

↓

map()
```

```
(Return Required Fields)

↓

Final API Response
```

Important Points

- Variables can make complex transformations much cleaner.
- Business logic is usually implemented before building the final response.
- Multiple DataWeave functions are often used together in real-world projects.
- Calculations such as total revenue are commonly performed using reduce().

Common Mistake

Many developers perform multiple transformations on the same payload repeatedly.

Instead of:

```
payload filter ...

payload filter ...

payload filter ...
```

Store the transformed result in a variable and reuse it.

This makes your DataWeave script:

- Cleaner
- Easier to debug
- Easier to maintain

Interview Tip

A common MuleSoft interview question is:

> Which DataWeave functions are commonly used in payment and order processing integrations?

A practical answer is:

> filter(), map(), distinctBy(), orderBy(), reduce(), default, and conditional logic are commonly used while processing order and payment-related data.

Pro Tip

When you receive a business requirement, don't immediately think about the code. Break it down into smaller steps first.

For example:

```
Requirement

↓

Filter the records

↓

Remove duplicates

↓

Sort the records

↓

Perform calculations

↓

Build the response payload
```

If you can do this mentally, writing the DataWeave code becomes much easier.

Topic 36 - Real-Time Scenario 3 (Employee Management API)

Time to Read: 6 Minutes

Problem Statement

An Employee Management API returns the following payload.

Your requirements are:

- Return only active employees.
- Group employees by department.
- Calculate the total number of active employees.
- Return the employee names in uppercase.
- Sort employees by their names.

This is a very common reporting requirement in enterprise applications.

Input

```
[
    {
        "employeeId": 101,
        "employeeName": "john doe",
        "department": "IT",
        "isActive": true
    },
    {
        "employeeId": 102,
        "employeeName": "emma wilson",
        "department": "HR",
        "isActive": true
    },
    {
        "employeeId": 103,
        "employeeName": "michael smith",
        "department": "IT",
        "isActive": true
    },
    {
        "employeeId": 104,
```

```
        "employeeName": "alice johnson",
        "department": "Finance",
        "isActive": false
    }
]
```

Functions Used

- filter()
- map()
- upper()
- orderBy()
- sizeOf()

Code

```
%dw 2.0
output application/json

var activeEmployees =
    payload
        filter ($.isActive)
        orderBy $.employeeName

---
{
    totalEmployees: sizeOf(activeEmployees),

    employees:
        activeEmployees map {
            employeeId: $.employeeId,
            employeeName: upper($.employeeName),
            department: $.department
        }
}
```

Output

```
{
    "totalEmployees": 3,
```

```
    "employees": [
        {
            "employeeId": 102,
            "employeeName": "EMMA WILSON",
            "department": "HR"
        },
        {
            "employeeId": 101,
            "employeeName": "JOHN DOE",
            "department": "IT"
        },
        {
            "employeeId": 103,
            "employeeName": "MICHAEL SMITH",
            "department": "IT"
        }
    ]
}
```

Bonus Example - Group Employees by Department

Code

```
%dw 2.0
output application/json
---
payload
    filter ($.isActive)
    groupBy $.department
```

Output

```
{
    "IT": [
        {
            "employeeId": 101,
            "employeeName": "john doe",
            "department": "IT",
            "isActive": true
        },
        {
            "employeeId": 103,
            "employeeName": "michael smith",
```

```
            "department": "IT",
            "isActive": true
        }
    ],
    "HR": [
        {
            "employeeId": 102,
            "employeeName": "emma wilson",
            "department": "HR",
            "isActive": true
        }
    ]
}
```

Important Points

- Reporting APIs often require multiple transformations.
- Variables help make complex logic more readable.
- groupBy() is commonly used for categorizing records.
- sizeOf() is useful for summary information.

Common Mistake

Many developers try to perform everything in a single long expression.

Instead:

```
Break the problem into smaller steps.

↓

Store intermediate results in variables.

↓

Build the final response.
```

Readable DataWeave is always better than clever DataWeave.

Interview Tip

A common MuleSoft interview question is:

> How do you generate summary information along with transformed records?

Answer:

> By combining functions such as filter(), sizeOf(), reduce(), and variables, we can easily build summary information along with the final API response.

Pro Tip

Enterprise APIs rarely return only transformed records. Most APIs also include:

- Total counts
- Summary information
- Status messages
- Grouped or categorized data

Learning how to combine transformed records with summary information is a valuable DataWeave skill.

Topic 37 - Real-Time Scenario 4 (Salesforce Account & Contact Transformation)

Time to Read: 6 Minutes

Problem Statement

Your MuleSoft application receives Account records from Salesforce.

Your requirements are:

- Return only Active Accounts.
- If the email is missing, use a default email address.
- Convert Account names to uppercase.
- Return only the required fields.
- Sort the Accounts by Account Name.

This is a very common Salesforce integration requirement.

Input

```
[
    {
        "Id": "001ABC001",
        "Name": "Global Technologies",
        "Email": "support@globaltech.com",
        "Status": "Active"
    },
    {
        "Id": "001ABC002",
        "Name": "Tech Solutions",
        "Status": "Active"
    },
    {
        "Id": "001ABC003",
        "Name": "Enterprise Systems",
        "Email": "contact@enterprise.com",
        "Status": "Inactive"
    }
]
```

Functions Used

- filter()
- orderBy()
- map()
- upper()
- default()

Code

```
%dw 2.0
output application/json
---
payload
    filter ($.Status == "Active")
    orderBy $.Name
    map {
        accountId: $.Id,
        accountName: upper($.Name),
        email: $.Email default "support@example.com"
    }
```

Output

```
[
    {
        "accountId": "001ABC001",
        "accountName": "GLOBAL TECHNOLOGIES",
        "email": "support@globaltech.com"
    },
    {
        "accountId": "001ABC002",
        "accountName": "TECH SOLUTIONS",
        "email": "support@example.com"
    }
]
```

What's Happening?

```
Salesforce Payload

↓

Filter Active Accounts

↓

Sort by Account Name

↓

Handle Missing Email Values

↓

Transform Required Fields

↓

Return Final API Response
```

Important Points

- Salesforce payloads are usually Arrays of Objects.
- The default keyword is extremely useful when Salesforce fields are empty or missing.
- Field names in Salesforce are case-sensitive.
- DataWeave transformations for Salesforce integrations are usually simple but very frequent.

Common Mistake

Many developers write unnecessary transformations when Salesforce already provides the required data.

Always ask yourself:

> "Do I really need to transform this field?"

If not, simply return it as it is.

Good DataWeave transformations are simple and readable.

Interview Tip

A common MuleSoft interview question is:

> Which DataWeave functions do you use most frequently in Salesforce integrations?

A practical answer is:

> filter(), map(), default, if-else, orderBy(), distinctBy(), and String functions are commonly used while transforming Salesforce payloads.

Pro Tip

If you're working with Salesforce integrations, master these DataWeave concepts really well:

- map()
- filter()
- default
- if-else
- orderBy()
- distinctBy()
- String functions
- Date transformations

These concepts alone can solve most day-to-day Salesforce transformation requirements.

Topic 38 - Top DataWeave Interview Questions

Time to Read: 8 Minutes

> If you're short on time before an interview, read this section at least once.

---

1. What is DataWeave?

Answer:

> DataWeave is MuleSoft's powerful transformation language used to transform data between different formats such as JSON, XML, CSV, Java Objects, and more.

---

2. What is the difference between map() and mapObject()?

Answer:

Function	Works With
map()	Arrays
mapObject()	Objects

---

3. What is the difference between filter() and distinctBy()?

Answer:

> `filter()` removes unwanted records based on a condition.
>
> `distinctBy()` removes duplicate records.

---

4. What is the difference between default and if-else?

Answer:

> Use `default` for handling null or missing values.
>
> Use `if-else` when applying business logic or multiple conditions.

---

5. Which DataWeave function is used for sorting records?

Answer:

`orderBy()`

---

6. Which DataWeave function is used for grouping records?

Answer:

`groupBy()`

---

7. Which DataWeave function is used for calculating totals?

Answer:

`reduce()`

Examples:

- Total Revenue
- Total Salary
- Total Orders

---

8. Which DataWeave function is used for handling empty payloads?

Answer:

`isEmpty()`

---

9. Which DataWeave function is used for removing nested Arrays?

Answer:

`flatten()`

---

10. Which DataWeave function is used for converting an Object into an Array?

Answer:

`pluck()`

---

11. Can DataWeave transform XML into JSON?

Answer:

Yes. XML to JSON transformation is very common in MuleSoft projects.

---

12. Can DataWeave work with CSV files?

Answer:

Yes. DataWeave can easily transform CSV into JSON and vice versa.

---

13. What is the purpose of the update operator?

Answer:

The update operator allows us to modify specific fields in an existing payload without rebuilding the entire Object.

---

14. Which DataWeave function is used for validating the number of records?

Answer:

`sizeOf()`

---

15. Which DataWeave functions do you use most frequently in real projects?

Answer:

The most commonly used functions are:

- map()
- filter()
- default
- if-else
- orderBy()
- distinctBy()
- reduce()
- sizeOf()

- isEmpty()
- upper()
- lower()
- trim()
- splitBy()
- replace()

Most Asked Interview Question

> Which DataWeave functions should I master if I have only one day before my interview?

Master these first:

```
1. map()

2. filter()

3. default

4. if-else

5. orderBy()

6. distinctBy()

7. reduce()

8. mapObject()

9. isEmpty()

10. groupBy()

11. XML Transformations

12. Date Transformations
```

If you're comfortable with the above topics, you'll be able to answer a large percentage of DataWeave interview questions confidently.

Interview Pro Tip

Whenever an interviewer asks:

> "Which DataWeave function would you use here?"

Don't immediately answer with a function name.

Instead:

1. Understand the business requirement.
2. Break it into smaller steps.
3. Explain your approach.
4. Then mention the appropriate DataWeave functions.

Interviewers usually evaluate your problem-solving approach more than your ability to memorize functions.

Golden Rule for DataWeave

Remember this simple mapping:

```
Arrays?

↓

map()
filter()
orderBy()
distinctBy()
reduce()


Objects?

↓

mapObject()
pluck()
update()


Null Values?

↓

default
isEmpty()
```

```
Strings?

↓

upper()
lower()
trim()
replace()
splitBy()
joinBy()


Reports & Summary?

↓

groupBy()
reduce()
sizeOf()


Dates?

↓

now()
Date Formatting


Files?

↓

JSON
XML
CSV
```

If you remember this one page, you'll already know which DataWeave function to use for most real-world requirements.

Topic 39 - Top 25 Tricky DataWeave Scenarios

Time to Read: 8 Minutes

> Don't memorize the code. Understand which DataWeave functions you would use to solve the problem.

Scenario 1

Requirement:

- Return only active customers.
- Remove duplicate customer IDs.
- Sort the customers alphabetically.

Functions to Use:

```
filter()
distinctBy()
orderBy()
```

Scenario 2

Requirement:

- Calculate the total revenue from successful orders.

Functions to Use:

```
filter()
reduce()
```

Scenario 3

Requirement:

- Replace missing customer countries with "India".

Functions to Use:

```
default
```

Scenario 4

Requirement:

- Convert customer names to uppercase and remove unwanted spaces.

Functions to Use:

```
trim()
upper()
```

Scenario 5

Requirement:

- Group employees by department.

Functions to Use:

```
groupBy()
```

Scenario 6

Requirement:

- Convert a comma-separated String into an Array.

Functions to Use:

```
splitBy()
```

Scenario 7

Requirement:

- Convert an Array of roles into a comma-separated String.

Functions to Use:

```
joinBy()
```

Scenario 8

Requirement:

- Remove duplicate email addresses from an API response.

Functions to Use:

```
distinctBy()
```

Scenario 9

Requirement:

- Check whether an API returned any records.

Functions to Use:

```
isEmpty()
sizeOf()
```

Scenario 10

Requirement:

- Calculate the total number of successful transactions.

Functions to Use:

```
filter()
sizeOf()
```

Scenario 11

Requirement:

- Convert XML into JSON.

Functions to Use:

```
DataWeave XML Transformation
```

Scenario 12

Requirement:

- Convert JSON into XML.

Functions to Use:

```
output application/xml
```

Scenario 13

Requirement:

- Format a date received from an external API.

Functions to Use:

```
as Date

as String {format: "..."}
```

Scenario 14

Requirement:

- Modify only one field in a large payload.

Functions to Use:

```
update
```

Scenario 15

Requirement:

- Build a welcome message for Premium customers.

Functions to Use:

```
if-else

or

match expression
```

Scenario 16

Requirement:

- Return only required fields from an API response.

Functions to Use:

```
map()
```

Scenario 17

Requirement:

- Flatten nested Arrays received from multiple systems.

Functions to Use:

```
flatten()
```

Scenario 18

Requirement:

- Convert Object values into an Array.

Functions to Use:

```
pluck()
```

Scenario 19

Requirement:

- Sort employees by salary.

Functions to Use:

```
orderBy()
```

Scenario 20

Requirement:

- Calculate the total salary of all employees.

Functions to Use:

```
reduce()
```

Scenario 21

Requirement:

- Check whether a country is supported by your API.

Functions to Use:

```
contains()
```

Scenario 22

Requirement:

- Process CSV data received from an SFTP server.

Functions to Use:

```
filter()
map()
orderBy()
```

Scenario 23

Requirement:

- Return a friendly response if the payload is empty.

Functions to Use:

```
isEmpty()
if-else
```

Scenario 24

Requirement:

- Rename Object fields dynamically.

Functions to Use:

```
mapObject()
```

Scenario 25

Requirement:

- Build an API response that contains:
  - Total Records
  - Summary Information
  - Transformed Data

Functions to Use:

```
sizeOf()
reduce()
map()
```

Interview Tip

If you can identify which DataWeave functions are required for a business problem, you've already solved most of the challenge.

Interviewers are often more interested in:

- Your approach.
- Your understanding of the payload.
- Your choice of DataWeave functions.

Golden Formula

```
Understand the Payload

↓

Understand the Requirement

↓

Select the Right Functions

↓

Build the Transformation

↓

Return the Final Response
```

This is exactly how experienced MuleSoft developers approach DataWeave transformations in real-world projects.

Topic 40 - Common DataWeave Mistakes Developers Make

Time to Read: 6 Minutes

> Learn from these mistakes once. Avoid them forever.

Mistake 1

Using map() on an Object

Incorrect:

```dw id="a4jm71"
payload map ...
```

Correct:

```dw id="sn64zf"
payload mapObject ...
```

Remember:

```text id="1whhpy"
Array -> map()

Object -> mapObject()
```

Mistake 2

Forgetting to Handle Null Values

Incorrect:

```dw id="bpmjj2"
payload.country
```

Better:

```dw id="aqoznt" payload.country default "India"
```

```
Always assume external APIs can send incomplete data.

---

## Mistake 3

### Writing Nested if-else Statements Everywhere

Instead of:

```text id="k1epmf"
if

↓

else if

↓

else if

↓

else
```

Consider:

```text id="xfmpn7" match expression
```

```
Cleaner code is easier to maintain.

---

## Mistake 4

### Not Checking Empty Payloads

Incorrect:

```dw id="4jivji"
payload map ...
```

Better:

```dw id="nljmyx" if (isEmpty(payload))

```
Never assume that APIs always return data.

Mistake 5

Writing Multiple Transform Message Components

Instead of:

```text id="1yx1gi"
Transform Message

↓

Transform Message

↓

Transform Message
```

Prefer:

```text id="mm3ag7" Single Transformation

↓
```

Cleaner Logic

```
Keep related transformations together whenever possible.

---

## Mistake 6

### Repeating the Same Logic Multiple Times

Incorrect:

```text id="k6ppye"
```
```

```
payload filter ...

payload filter ...

payload filter ...
```

Better:

```dw id="n6ah95" var activeCustomers = ...
```

```
Use variables whenever they improve readability.

---

## Mistake 7

### Forgetting That groupBy() Returns an Object

Many developers expect:

```text id="znchyr"
Array
```
```

Actual output:

```text id="gy2vr1" Object
```

```
This is one of the most frequently asked MuleSoft interview questions.

---

## Mistake 8

### Ignoring Date Formats

APIs often use different formats:

```text id="pbv7lr"
yyyy-MM-dd

dd-MM-yyyy

MM/dd/yyyy
```
```

Always validate the expected format before performing transformations.

Mistake 9

Using Complex DataWeave for Simple Problems

Requirement:

> Provide a default country value.

Incorrect approach:

```dw id="4etmfu" if (payload.country != null) ...

Better:

```dw id="z7f9fz"
payload.country default "India"
```

Simple code is usually better code.

Mistake 10

Not Understanding the Payload Structure

Before writing any DataWeave code, always ask:

```text id="1sh7oc" Is it an Object?

↓

Is it an Array?

↓

Is it an Array of Objects?

↓

Is it XML?

↓

Is it CSV?

````
This habit alone will save you countless debugging hours.

Top 5 Interview Mistakes

Developers often struggle with:

1. map() vs mapObject()
2. groupBy() return type.
3. reduce() syntax.
4. XML transformations.
5. Date formatting.

Spend extra time mastering these topics.

Golden Rules

```text id="e4oed6"
Always use default for null values.

Always check the payload structure.

Always validate empty payloads.

Prefer readability over clever code.

Use variables when transformations become complex.

Keep transformations simple.
````

Pro Tip

When debugging DataWeave issues in real projects, 80% of the problems are usually caused by:

- Incorrect payload assumptions.
- Null values.
- Wrong function selection.
````

- Unexpected API responses.
- Data format mismatches.

Whenever a transformation isn't working, the first thing you should inspect is the payload itself.

One-Line Advice

> Never start writing DataWeave code before understanding the payload.

That one habit will make you a much better MuleSoft developer.

Topic 41 - DataWeave Cheat Sheet

Bookmark this page. You'll come back to it often.

Arrays

Requirement	Function
Transform records	map()
Filter records	filter()
Remove duplicates	distinctBy()
Sort records	orderBy()
Calculate totals	reduce()
Group records	groupBy()
Flatten nested arrays	flatten()

Objects

Requirement	Function
Transform object fields	mapObject()
Convert Object to Array	pluck()
Update specific fields	update

Strings

Requirement	Function
Convert to Uppercase	upper()
Convert to Lowercase	lower()
Remove spaces	trim()
Capitalize first letter	capitalize()
Replace text	replace()

Requirement	Function
Split String	splitBy()
Join Array values	joinBy()
Check value exists	contains()

Validations

Requirement	Function
Check null or missing values	default
Check empty payload	isEmpty()
Count records	sizeOf()
Apply conditions	if-else
Multiple conditions	match expression

Dates & Time

Requirement	Function
Current Date & Time	now()
Format Dates	as Date
Convert Date Formats	as String

File Transformations

Requirement	Output Type
JSON Transformation	application/json
XML Transformation	application/xml
CSV Transformation	application/csv

Most Used DataWeave Functions in Real Projects

```
1. map()
2. filter()
3. default
4. if-else
5. distinctBy()
6. orderBy()
7. reduce()
8. mapObject()
9. sizeOf()
10. isEmpty()
11. groupBy()
12. upper()
13. trim()
14. splitBy()
15. replace()
16. update
17. flatten()
18. pluck()
19. now()
20. match expression
```

Golden Formula

```
Business Requirement

↓

Understand the Payload

↓

Choose the Correct Function

↓

Transform the Data

↓

Validate the Output
```

```
↓

Return the Final Payload
```

Function Selection Guide

```
Need to Transform Records?

↓

map()


Need to Filter Records?

↓

filter()


Need to Remove Duplicates?

↓

distinctBy()


Need Total Count?

↓

sizeOf()


Need Total Amount?

↓

reduce()


Need Default Values?

↓
```

```
default


Need Conditional Logic?

↓

if-else or match


Need XML or CSV Transformation?

↓

DataWeave Supports Both


Need to Modify Existing Payload?

↓

update
```

Interview Day Quick Revision

Before your interview, make sure you're comfortable with:

- map()
- filter()
- default
- reduce()
- orderBy()
- distinctBy()
- mapObject()
- groupBy()
- update
- XML Transformations
- Date Formatting
- CSV Transformations
- match expression
- Real-Time Scenarios

If you can confidently explain and write examples for the above topics, you're well prepared for most MuleSoft DataWeave interview questions.

Topic 42 - Top 20 Most Used DataWeave Functions in Real Projects

> If you master these functions, you'll be able to handle most day-to-day MuleSoft transformations confidently.

Top 20 Functions (Ranked)

Rank	Function	Usage Level
1	map()	Very High
2	filter()	Very High
3	default	Very High
4	if-else	Very High
5	orderBy()	High
6	distinctBy()	High
7	reduce()	High
8	mapObject()	High
9	isEmpty()	High
10	sizeOf()	High
11	groupBy()	Medium
12	upper()	Medium
13	lower()	Medium
14	trim()	Medium
15	splitBy()	Medium
16	replace()	Medium
17	update	Medium
18	flatten()	Medium
19	pluck()	Medium
20	now()	Medium

Must Learn for Beginners

Start with these topics first:

```text id="a72mg1"
1. Variables

1. Arrays

2. Objects

3. Payload

4. map()

5. filter()

6. default

7. if-else

8. orderBy()

9. distinctBy()
```

```
If you understand these concepts well, you'll already be able to solve many
common transformation requirements.

---

## Must Learn for Intermediate Developers

```text id="m81xh4"
1. reduce()

2. mapObject()

3. groupBy()

4. flatten()

5. sizeOf()

6. isEmpty()

7. splitBy()
```
```

```
8. replace()

9. Date Transformations

10. XML Transformations
```

These topics are commonly used in enterprise MuleSoft projects.

Must Learn for Interviews

Interviewers frequently ask questions on:

```text id="q52vn8" 1. map() vs mapObject()

1. filter()

2. reduce()

3. groupBy()

4. default

5. XML Transformations

6. CSV Transformations

7. update Operator

8. Date Formatting

9. Real-Time Scenarios
```

```
---

## Most Common Function Combinations

### Combination 1

```text id="v71op3"
filter()

↓
```
```

```
map()
```

Used for:

- Customer APIs
- Employee APIs
- Salesforce Integrations

Combination 2

```text id="n27fz6" filter()

↓

distinctBy()

↓

orderBy()

↓

map()
```

```
Used for:

- API Response Transformations
- Reporting APIs

---

### Combination 3

```text id="r61ud9"
filter()

↓

reduce()
```
```

Used for:

- Total Revenue
- Total Salary
- Total Orders

Combination 4

```text id="t43bc8" trim()

↓

replace()

↓

default
```

```
Used for:

- Data Cleaning
- Input Validations

---

### Combination 5

```text id="k18wd7"
XML

↓

DataWeave Transformation

↓

JSON
```

Used for:

- SOAP Integrations
- Legacy Systems

```

Real Project Tip

If you're working as a MuleSoft developer, you'll probably use the following functions almost every week:

- map()
- filter()
- default
- if-else
- orderBy()
- distinctBy()
- reduce()
- isEmpty()
- sizeOf()
- String functions

Master these first before moving to advanced topics.

80/20 Rule for DataWeave

```text id="x95kr4" Learn 20% of the functions.

↓

Solve 80% of the transformation problems. ```

You do NOT need to memorize every DataWeave function available. Focus on the functions that are commonly used in real-world integrations.

---

## Final Advice

If you ever feel overwhelmed while learning DataWeave, remember this:

> You don't become good at DataWeave by memorizing functions. You become good at DataWeave by understanding business requirements and choosing the right function for the job.

Keep practicing small transformations, combine functions together, and always start by understanding the payload structure.
```

Topic 43 - Quick Revision Notes (Interview Day)

Read this page once before your MuleSoft interview.

Always Remember

```
Array  -> map()

Object -> mapObject()

Object -> pluck() -> Array

Nested Arrays -> flatten()

Duplicates -> distinctBy()

Sorting -> orderBy()

Totals -> reduce()

Grouping -> groupBy()
```

Null and Empty Values

```
Null or Missing Value?

↓

default


Empty Payload?

↓

isEmpty()


Count Records?
```

```
↓

sizeOf()
```

Conditional Logic

```
Simple Conditions?

↓

if-else


Multiple Conditions?

↓

match expression
```

String Functions

```
UPPERCASE

↓

upper()


lowercase

↓

lower()


Remove Spaces

↓

trim()
```

```
Replace Values

↓

replace()


Split Values

↓

splitBy()


Join Values

↓

joinBy()
```

Date Functions

```
Current Date & Time

↓

now()


Format Date

↓

as Date

↓

as String {format:"..."}
```

Most Important Functions for Interviews

Make sure you're comfortable writing examples for:

- map()
- filter()
- default
- if-else
- distinctBy()
- orderBy()
- reduce()
- mapObject()
- groupBy()
- isEmpty()
- sizeOf()
- update
- XML Transformations
- CSV Transformations
- Date Formatting

Top Interview Questions

Be prepared to answer:

1. What is DataWeave?
2. What is the difference between map() and mapObject()?
3. What is the difference between filter() and distinctBy()?
4. When do we use reduce()?
5. What is groupBy() and what does it return?
6. What is the use of the default keyword?
7. What is the update operator?
8. How do you transform XML to JSON?
9. How do you format dates in DataWeave?
10. Which DataWeave functions do you use in real projects?

Common Function Combinations

API Response Transformation

```
filter()

↓
```

```
distinctBy()

↓

orderBy()

↓

map()
```

Total Revenue Calculation

```
filter()

↓

reduce()
```

Input Validation

```
trim()

↓

default

↓

if-else
```

Summary Response

```
sizeOf()

↓

reduce()

↓

map()
```

Before Writing Any DataWeave Code

Always ask yourself:

```
1. What is my payload?

↓

2. What is my requirement?

↓

3. Which DataWeave functions should I use?

↓

4. What should my final output look like?
```

This simple approach will help you solve most DataWeave problems.

Final Interview Advice

Never start coding immediately.

In interviews:

- Understand the payload.
- Explain your approach.
- Mention the functions you'll use.
- Then write the DataWeave code.

Interviewers appreciate clear thinking more than memorized answers.

The One Thing You Should Never Forget

> DataWeave is not about memorizing functions. It's about understanding the data, understanding the business requirement, and choosing the right transformation approach.

If you understand those three things, you've already mastered the most important part of DataWeave.

Thank You!

Congratulations on completing this DataWeave eBook.

By reaching this page, you've learned the concepts that MuleSoft developers use every day in real-world integrations, including:

- Core DataWeave concepts.
- Arrays and Objects.
- DataWeave functions.
- XML and CSV transformations.
- Date and String manipulations.
- Real-time API scenarios.
- Interview preparation topics.
- Quick revision notes and cheat sheets.

Remember, becoming good at DataWeave isn't about memorizing every function. It's about understanding:

- The payload.
- The business requirement.
- The right transformation approach.

Whenever you're stuck, ask yourself three simple questions:

1. What data am I receiving?
2. What data do I need to return?
3. Which DataWeave function can help me get there?

Keep your transformations:

- Simple.
- Readable.
- Maintainable.

A Small Piece of Advice

Don't try to learn everything in a single day.

Practice small transformations regularly. Build confidence by solving real-world problems, and over time, you'll naturally become comfortable with DataWeave.

Your DataWeave Learning Journey

```
Understand the Payload

↓
```

```
Understand the Requirement

↓

Choose the Right Function

↓

Transform the Data

↓

Validate the Output

↓

Build Better Integrations
```

Final Words

DataWeave is one of the most powerful and enjoyable parts of MuleSoft. The more you practice, the more elegant and efficient your transformations will become.

Keep learning. Keep building. Keep transforming data.

Happy Coding!